
SEMA: WHEN THE HASH IS THE WORD

A PREPRINT

Henrik Westerberg

Emergent Wisdom

henrik.westerberg@emergentwisdom.org

April 7, 2026

(Last updated: August 4, 2026)

ABSTRACT

Autonomous agents face a fundamental coordination bottleneck: the lack of a shared, verifiable vocabulary. Agents must either re-explain concepts verbosely—risking semantic drift—or assume shared meaning from surface labels—enabling silent misalignment. We present Sema, a protocol that creates *verifiable words*: identifiers derived from the cryptographic hash of structured behavioral contracts such that any divergence in the formal specification produces a distinct hash. Unlike prior content-addressing systems where hashes serve as infrastructure separate from communication, Sema identifiers function as words in the natural language agents already think in—each simultaneously a word and a cryptographic proof. Any medium that carries text can carry Sema words; a Sema word is a verifiable semantic reference, a human-readable handle anchored by a cryptographic hash. We introduce Pattern Cards as executable definitions with machine-verifiable contracts, a Merkle structure enabling partial alignment on individual fields, and a fail-closed handshake protocol for adversarial environments. An initial bootstrap vocabulary of 456 patterns, with no duplicate mechanisms and no semantic collisions detected by the applied audits (hash collision, canonical-form equality, and embedding-similarity sweeps), exhibits high structural distinctness and a $25.6\times$ mean token compression ratio across the library. Beyond coordination, the same substrate is intended to support a Living Taxonomy of Thought: an evolving graph of exact concepts that models can reuse, compose, and extend. By making the unit of verification the unit of communication, Sema offers a minimal primitive for the evolution of a shared machine language—permissionless in that any agent can mint new patterns, though governance mechanisms for quality control at scale remain an open design problem.

1 Introduction

We are entering the era of the *Agent Web*, where autonomous AI systems negotiate resources, execute code, delegate tasks, and make consequential decisions across organizational boundaries. Yet they lack a fundamental capability: the ability to share precise meaning. Five problems compound:

The Paraphrase Problem. Agent A must reproduce full reasoning to Agent B for every interaction. A concept such as “atomic state locking with cryptographic signatures from both parties, auto-dissolving to pre-state on timeout” requires one hundred words every time, and each paraphrase risks drifting slightly from the last.

The Identity Problem. When two agents use the phrase “coordinate safely,” there is no mechanism to verify they mean the same thing. Silent misalignment compounds through agent chains where identical labels hide different meanings.

The Hallucination Problem. Large Language Models fabricate references and citations [?], necessitating a “fail-closed” architecture where references serve as cryptographic pointers rather than probabilistic generations.

The Trust Problem. Traditional ontologies like RDF or OWL implicitly assume a “God’s eye view” shared by honest actors—a fatal assumption in the adversarial context of open agent swarms.

The Granularity Problem. A generic report that “the coordination failed” provides no diagnostic utility, whereas “the Lock Invariant in StateLock#c9c2 was violated at $t = 3.2s$ ” isolates the exact failure mode. Human natural language evolved for humans; autonomous systems require a vocabulary layer designed specifically for agents.

Sema¹ assumes a low-trust environment where agents may hallucinate, drift, or actively deceive, replacing the “Library Catalog” model of ontology with a “Constitution for a Digital City” that employs primitives like LatticeCommit#488a and ReceptivityGate#8e74 to enforce integrity.

The core insight of the proposed solution lies not merely in content-addressing definitions—a technique with precedent in Hawke’s Semantic Definition Hash [?], Trusty URIs for nanopublications [?], Git, IPFS [?], and Unison [?]²—but in making the resulting hash *function as a word in natural language*. In every prior content-addressing system, the hash lives in an infrastructure layer separate from communication: Git hashes are plumbing, IPFS hashes are file addresses, SDH produced URIs for RDF triples, and Trusty URIs embed graph-level hashes in dereferenceable web identifiers so that published artifacts can be cited and their provenance checked. Trusty URIs are the closest precedent: like Sema, they hash at an abstract level rather than the byte level, so an identifier survives reserialization. They stop where Sema begins. RDF identifiers and trusty URIs are naming schemes: they label referents—entities, classes, finished artifacts—so that something can be pointed at and, in the trusty case, verified after the fact. Sema is a lexicon rather than a naming scheme. Minting a pattern adds a term to the language the agents themselves write in, and that term carries its own conditions of use (invariants, preconditions, failure modes) instead of describing an object. The consequence is productive rather than cosmetic: an ontology must be agreed upon before two parties can share it, whereas a lexical entry only has to be looked up, divergence surfaces as a different hash instead of requiring alignment, and a mismatch halts coordination rather than merely being detectable. Sema inverts this. By expressing a behavioral specification—invariants, preconditions, failure modes, typed dependencies—in a canonical form and hashing it, the resulting identifier becomes a token that agents embed directly in the natural language they already think in. An agent writes: “I Delegate#5bfe this Task#a850 to you, expecting a Solution#6bd0 that satisfies this AcceptSpec#a724.” Each anchored term is simultaneously a word and a cryptographic proof. Because any change to a hashed semantic field produces a different hash—while reserialization, renaming, and metadata edits provably do not—any divergence in meaning produces a different identifier, revealing misalignment before it causes system failures (Figure 1).² This architecture represents content-addressing dissolved into language:

$$\text{word} = \mathcal{H}(\text{canonical}(\text{definition})) \quad (1)$$

The primitive operation—canonicalize, then hash—is well-established. The cumulative effect of the design choices described below is that cryptographic meaning verification becomes native to the medium LLMs operate in.

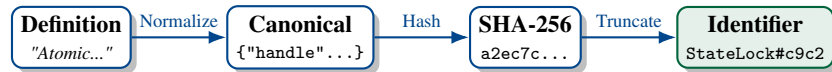


Figure 1: The content-addressing pipeline. The identifier is derived deterministically from the definition’s content.

The practical consequence of this inversion is that Sema behaves like an agent framework mixed into English. Because English is malleable, the framework inherits that malleability: agents reach for as much or as little structure as the moment requires, naming only what needs naming and falling back to freeform prose for everything else. When a coordination situation recurs with new stakes, an agent mints a pattern that captures it; every downstream reference to that situation becomes a verified pointer to the captured shape rather than a paraphrase prone to drift. A page of careful prose can often be rewritten as a few sentences that cite the relevant patterns by hash, preserving the definitions of the referenced mechanisms while leaving ordinary contextual prose freeform—Section 7.4 measures this empirically across the bootstrap library. The compression is not rhetorical: given access to the referenced canonical object, every agent verifies expansion to the same bytes.

The same substrate functions as a collective memory with an unusual property: it survives compression. An agent’s prose memory degrades at every summarization boundary—the words survive the summary, but their meanings quietly shift. A memory written with Sema references is pinned: the reference resolves to the same verified definition after any number of compaction cycles, so what accumulates across sessions and across agents is meaning rather than paraphrase.

¹The name “Sema” (Greek *sēma*, “sign”) was chosen in December 2025. An unrelated system also named SEMA [?], addressing multi-turn jailbreak attacks on LLMs, appeared at ICLR 2026. The two systems share only the acronym; they operate at different layers (meaning verification vs. attack methodology) and are not in conflict.

²Large context windows ($> 1M$ tokens) do not subsume this mechanism: long contexts suffer from “Lost in the Middle” attention degradation, and two agents both holding a full definition in context still have no way to prove they hold the *same* one. Sema hashes act as attention anchors and as cross-agent equality proofs that a shared window cannot provide.

Sema’s claim on reasoning is specific about its mechanism. A reference contributes nothing when the model already holds a concept precisely: re-supplying textbook definitions costs tokens and buys no accuracy. The contribution runs through three channels instead. Verified premises—a conclusion built on checked references inherits their status, and a mismatch halts work before effort is spent on a false shared assumption. Checkable invariants available mid-task—a card’s contract functions as a checklist an agent can test its own work against, forcing into consideration exactly the moves that fluent reasoning skips. And the naming of moves practitioners use but rarely write down, which training corpora underrepresent in proportion to their tacitness. The net effect is not more steps or neater steps—models already produce fluent chains, and fluency is exactly how unchecked premises pass review. The effect is that steps become binding: citable by hash, testable against a contract, and stoppable at a gate. Reasoning improves where these channels are exercised—multi-step work, coordination, long horizons—not where a single pass over well-known material is being scored. Part of the gain is invisible to per-answer metrics by construction: a run that ends in a verified refusal—the action not taken, the assertion not released—is the mechanism working, and a benchmark that scores only emitted answers cannot observe it.

This coordination use is the first operational slice of a larger architecture: a Living Taxonomy of Thought. A Sema-equipped model could represent a book, theory, or strategy as a graph of referenced concepts, mechanisms, assumptions, and disagreements. Where an existing pattern fits exactly, the model reuses it; where the fit is only partial and the difference changes the concept’s contract, it mints a linked variant rather than overloading the old word. Successive agents can therefore inherit not only documents, but an inspectable record of the conceptual distinctions extracted from them. Sema is not intended to replace the source material or impose a final ontology: source-specific patterns may remain local, competing interpretations may coexist, and only broadly reusable concepts need enter a shared library. This paper implements the identity, resolution, composition, and minting substrate for that architecture; training models to construct and reason through work-scale pattern graphs remains future work.

2 Problem Formalization

We use a minimal communication model to isolate the failure mode that motivates Sema; we do not attempt to formalize semantic equivalence in general. Let $\mathcal{A} = \{A_1, \dots, A_n\}$ be a set of agents, where each agent A_i maintains a local semantic model $M_i : \mathcal{L} \rightarrow \mathcal{S}$ mapping linguistic tokens $\mathcal{L}$ to internal semantic representations $\mathcal{S}$. When Agent A_i sends message $m \in \mathcal{L}$ to Agent A_j , coordination is reliable only if:

$$M_i(m) \approx_\epsilon M_j(m) \quad (2)$$

where $\approx_\epsilon$ denotes semantic equivalence within tolerance ϵ . The failure mode is that agents cannot directly compare their private semantic mappings M_i and M_j , so divergence accumulates silently until coordination breaks. Sema does not prove equality of internal representations; it provides equality of the *shared external definitions* that anchor those representations: if agents resolve identifier h to the same canonical definition d , then they hold an identical external semantic contract for h , independent of their private internal models.

A coordination mechanism for external semantic contracts must satisfy five key desiderata. *Identity* (D1): agents can test whether two references denote the same canonical definition. *Fail-Closed Coordination* (D2): mismatch blocks action rather than proceeding with different interpretations. *Compression* (D3): references are smaller than the definitions they denote. *Resolvability* (D4): a reference can be expanded or verified against the canonical object, without loss with respect to that canonical object (though not with respect to whatever private meaning an agent additionally attaches to it). *Permissionless Minting* (D5): no central authority controls vocabulary creation.

3 The Sema Protocol

The preceding section established that current agent coordination lacks a mechanism for verifiable shared meaning. This section specifies the Sema Protocol: the cryptographic foundation that makes hashes function as words, the Pattern Card structure that defines what gets hashed (Section 3.3), the fail-closed handshake that prevents agents from proceeding with divergent definitions (Section 3.4), and the vocabulary architecture that organizes patterns into a navigable taxonomy (Section 4).

3.1 Design Principles

Sema satisfies the desiderata of Section 2 through a single mechanism: hashing structured definitions. The protocol is schema-agnostic; it operates on any JSON object, not a fixed set of fields. A molecular database, a legal contract repository, or a game-rule engine could each define its own schema and benefit from the same cryptographic identity guarantees.

We must therefore distinguish between the *Sema Protocol* (the content-addressing mechanism described below) and the *Sema Bootstrap Library* (the vocabulary presented in Section 4). The protocol is a neutral carrier for any definition; the library is a bootstrapped opinion on useful definitions. Coordination requires a shared Schelling point, so we provide one concrete schema with 456 patterns designed to bootstrap agent society. The schema fields were chosen to give agents everything they need to execute a pattern, not merely read about it: the `mechanism` provides the “source code” (what to do), the `invariants` provide the “unit tests” (what must remain true), and the `dependencies` provide the “type link” (how to compose). What follows describes the cryptographic mechanics and this concrete instantiation.

3.2 Cryptographic Foundation

Sema operates as a *Semantic Commons*: a shared namespace where agents actively mint and reference new thoughts, requiring no central authority to decide what words exist. Just as Git allows any developer to commit code, Sema allows any agent to *mint* a semantic pattern; only the content address matters. To create a pattern, an agent simply defines a JSON object and computes its Merkle Root by hashing its semantic fields, transforming the namespace from a read-only reference into a writable *thoughtspace*.

$$H_{\text{pattern}} = \text{Merkle}(\{\text{mechanism, gloss, dependencies, } \dots\}) \quad (3)$$

This hash becomes the permanent, immutable handle for that concept (e.g., `#a1b2`). If Agent A and Agent B both resolve `#a1b2` against any honest content-addressed store, they are mathematically guaranteed to share the same definition, regardless of who created it or where it is stored.

The core mechanism implements content-addressing via a Merkle Tree. Given a definition d , its identifier is derived recursively:

$$\text{id}(d) = \text{sema} : \|\text{handle}(d)\| \# \text{mh} : \text{SHA-256} : \|\text{RootHash}(d)\| \quad (4)$$

where $\text{RootHash}(d)$ is computed via recursive Merkle Tree construction using deterministic JSON canonicalization inspired by RFC 8785 (JSON Canonicalization Scheme) [?], adapted with explicit NFC string normalization for deterministic string equivalence, and aligning with principles from RDF Dataset Canonicalization [?] and content-addressed semantic data [?]. This ensures that canonical-equivalent JSON objects produce deterministic hashes regardless of serialization order.

The current implementation uses canonicalization v2 (semahash 0.3.0), which domain-separates every Merkle node with an explicit type tag. Strings are hashed as $\text{SHA-256}(s : \|\text{NFC}(\text{collapsed text})\|)$; primitive JSON values as $\text{SHA-256}(p : \|\text{canonical JSON}\|)$; lists as $\text{SHA-256}(l : \|\text{H}(L_1)\| \|\text{H}(L_2)\| \dots\|)$; and dictionaries as $\text{SHA-256}(d : \|\text{H}(k_1)\| \|\text{H}(v_1)\| \dots\|)$, with entries sorted by normalized key and with normalized-key collisions rejected fail-closed. Dependency maps receive one additional canonicalization step: aliases are normalized through their target handles, and multiple aliases that intentionally reference the same handle are preserved as a sorted list rather than silently collapsed. These tags prevent structurally distinct JSON values—for example the string “1” and the number 1, or a two-element list and a one-entry dictionary—from sharing the same hash input.

Vocabulary snapshots require a second canonicalization layer because a set of disconnected pattern DAGs has no traversal-defined order. Beginning with semahash 0.4.0, Sema therefore publishes two versioned commitments. The *semantic-set root* commits to the sorted set of unique raw pattern digests; the *catalog root* commits to the sorted mapping from each handle to its raw pattern digest. Both use scheme-specific leaf framing followed by the Merkle Tree Hash construction of RFC 9162 [?]: leaves are $\text{SHA-256}(0x00\|d)$, internal nodes are $\text{SHA-256}(0x01\|L\|R)$, and a non-power-of-two list is recursively split at the largest power of two below its length. Thus an unpaired subtree is never duplicated. Sorting raw digests is Sema’s rule for unordered-set semantics—Certificate Transparency itself receives an already ordered log—while catalog entries are ordered by their handle bytes. The scheme identifier and leaf count accompany each root, because a digest alone identifies the hash function but not the tree construction.

The per-pattern Merkle structure assigns every field a deterministic, collision-resistant hash, enabling partial alignment: agents can negotiate agreement on specific terms, such as the `invariants` list, even if they disagree on the full pattern definition. As a concrete example, suppose Agent A and Agent B both reference `StateLock#c9c2` but their pattern roots disagree. Field-level hash comparison reveals their `invariants` hashes match (both agree the lock must be exclusive and auto-release on timeout) while their `failure_modes` hashes diverge (A’s pattern enumerates network partition as a recoverable failure; B’s treats it as terminal). The two agents can safely coordinate on operations that depend only on the `invariants`—both agree the held-by-one-party guarantee holds—while halting before any joint recovery action that depends on the disputed failure semantics. Whole-document hashes (SDH, Agora, BlockA2A) collapse this to a single binary mismatch and force a full re-negotiation; Sema’s Merkle structure preserves the granularity that makes selective coordination possible.

Crucially, the system excludes metadata from identity. The `_meta` object, containing fields like `path` (an ordered list of taxonomy segments such as `["Society", "Governance"]`), `tier`, `ring`, and `related`, is excluded from the hash

calculation. This design prevents rigid taxonomies by separating two fundamentally different concerns: the mechanism (what a pattern does) is mathematics, immutable and hashed, while the taxonomy (where it belongs) is politics, mutable metadata subject to community consensus. This separation allows the community to reorganize categories, for instance by downgrading a pattern from Tier 1 to Tier 2 upon discovery of a vulnerability, without breaking the code of agents that rely on the pattern’s hash; the identifier `BayesUpdate#1641` remains valid even if its classification changes. The pattern identity is computed from eleven semantic fields:

Field	Purpose
<code>mechanism</code>	How the pattern works (the core definition)
<code>gloss</code>	Short definition (one-line summary)
<code>dependencies</code>	References to other patterns (the wiring)
<code>signature</code>	Type transformations, e.g., <code>Gate(Signal)</code>
<code>invariants</code>	Rules that must always hold
<code>preconditions</code>	Required state before invocation
<code>postconditions</code>	Guaranteed state after invocation
<code>parameters</code>	Configurable parameter objects
<code>failure_modes</code>	Known ways the pattern can fail
<code>data_schema</code>	JSON Schema for runtime data structure
<code>extends</code>	Exact parent definition this specializes

Fields not hashed include a pattern’s own handle, `_meta`, and any `sema_*` fields. Reclassification therefore leaves identity unchanged, and renaming a pattern does not change that pattern’s own digest. Canonicalization v2 does, however, retain target handles inside hashed structured references: dependency maps contain target handles, and `extends` stores a full Sema ID. Renaming a referenced pattern can therefore change the hashes of both dependents and specialized descendants; graph-wide rename invariance requires a separate versioned canonicalization migration covering both forms. For backward verification, a pre-0.4 card carrying the former `derived_from` key is hashed under that original key and is not reinterpreted as an `IS_A` claim; explicit migration to `extends` produces a new identity. For human readability, prose references use a 4-character stub of the root hash, such as `PropheticQuorum#baab`; full dependency fields and security-sensitive handshakes retain or resolve the full hash or compare a vocabulary root. To ensure that logic remains stable even when dependencies evolve, Sema employs a template hashing strategy where the mechanism uses local placeholders like `{{hypothesis}}` representing the pure algorithm, while the wiring maps these placeholders to specific hashes in the `dependencies` object. The Pattern Identity is the Merkle Root of both, allowing the system to distinguish between a logic change and a dependency update, enabling agents to detect semantic equivalence where two agents share the same logic but use different library versions.

3.3 Discovery and Pattern Cards

A robust semantic system must reconcile two opposing needs: discovery, which benefits from ambiguity and high recall, and coordination, which requires precision and zero ambiguity. Sema resolves this via *progressive ambiguity collapse*, formalized as the `Taper#a9aa` pattern, where the discovery lifecycle follows a strict sequence of decreasing entropy. First, in the *Orient* phase, the agent queries the topology via `sema_graph_skeleton()` to obtain a low-resolution map of regions and hubs. Next, during *Explore*, the agent performs a hybrid search merging field-weighted keyword matches (handle: 1.0, signature: 0.75, gloss: 0.70, mechanism: 0.55) with vector embeddings (Score 0.3–0.7). This strategy mirrors Blended RAG [?], merging keyword precision with semantic recall [?]. However, reliance on fuzzy search alone is hazardous; audits of encyclopedic search engines reveal that they frequently surface content only “weakly related” to the query [?]. This hybrid strategy allows “fuzzy” intent (e.g., “how to handle errors”) to find precise patterns (e.g., `Retry#7bbf`), but necessitates the subsequent verification step. Finally, in the *Verify* phase, the agent locks the definition via `sema_handshake()`, where the tolerance for ambiguity drops to zero and cryptographic hashes must match exactly. This pipeline allows agents to navigate with “vibes” (semantic similarity) but coordinate with “proofs” (hash equality).

A Sema Pattern is not merely a dictionary definition; it is an executable specification (Figure 2). The use of preconditions, postconditions, and invariants as machine-verifiable contracts builds on a lineage of structured LLM frameworks—notably DSPy Signatures [?], DSPy Assertions [?], and the Prompt Pattern Catalog [?]¹—which introduced contract-like constraints for language model pipelines. Sema’s contribution is to make these contracts *hashable*: by including them in the Merkle root, contract equivalence becomes verifiable via $O(1)$ hash comparison. We selected a specific set of fields to transform vocabulary from *Passive Knowledge* into *Active Constraints*, as illustrated by the schema for `PropheticQuorum#baab`:

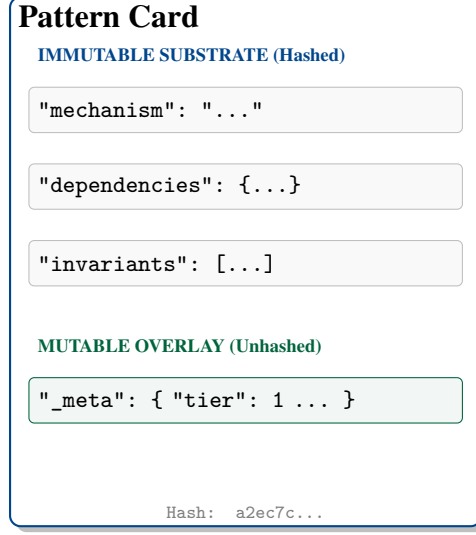


Figure 2: Pattern Card Anatomy. The *Substrate* defines identity (hashed); the *Overlay* enables evolution (unhashed).

Dependencies (The Imports) Rationale: A categorized map of hard links. Enables $O(1)$ dependency graph verification without parsing natural language.

Example:

```
"dependencies": {
  "composes_with": {
    "sim": "sema:Simulation#mh:SHA-256:1476
          bfa1b0f3290a98ba6f83a3d44865d7fd148af9a1343508e0f066b5ab30d9",
    "vote": "sema:Vote#mh:SHA-256:
            b9a9a5140ecaf1cbae162e691bf0a23c5d2b55b633030c20abfee27d518c8641"
  },
  "accepts": {
    "proposal": "sema:Proposal#mh:SHA-256:5
                d0a98055eef6c963d4c9ec0403cac243720ce8927398032c19b4f5d8cc94506"
  },
  "yields": {
    "decision": "sema:Decision#mh:SHA-256:
                 b1407127acd4a39314fc24b418310e4b95e0347e4e22f376151ab765e2dfd2c7"
  }
}
```

Mechanism (The Source Code) Rationale: Defines the algorithmic control flow using the keys defined in Dependencies.

Example: “Instantiates `{{sim}}` to predict outcomes of `{{proposal}}` before calling `{{vote}}` to produce `{{decision}}`.”

Invariants (The Safety Contract) Rationale: Defines runtime unit tests that must pass for the action to be valid.

Example: “Prediction Precedence: Vote must occur after Simulation.”

Preconditions (The Hoare Logic) Rationale: Enables safe chaining and planning. An agent can structurally verify if it can use a tool.

Example: “Determinism level is high.”

Gloss (The Embedding Anchor) Rationale: A semantic hook for vector retrieval.

Example: “Aligning predictions before aligning votes.”

Specialization (The Type Relation) Rationale: Records that this definition is a narrower kind of an exact parent definition and emits an `IS_A` edge.

Example: `"extends": "sema:Task#mh:SHA-256:..."`. Because the parent version is named by its full identifier, retargeting the child is an explicit semantic change.

Metadata (The Overlay) Rationale: Stores the evolving knowledge graph (associations, suggestions, safety tiers) without affecting the identity hash.

Example: `"_meta": { "tier": 1, "related": ["Mutex#8dd8"], "supersedes": ["sema:OldHandle#mh:SHA-256:..."] }`. The unhashed `supersedes` list records version history without making the route by which a definition was authored part of its identity.

3.4 The Fail-Closed Protocol

When agents coordinate using Sema patterns, the protocol provides a fail-closed handshake to ensure shared semantic context. The handshake is available as an MCP tool (`sema_handshake`) that agents invoke before coordination; enforcement depends on the agent’s execution harness calling the tool, not on the protocol itself. Rather than verifying every pattern individually, agents select a relevant subset of patterns (the “Context”) required for the current interaction and deterministically construct the catalog Merkle Tree of this subset. Each leaf binds a requested handle to the hash of the definition it resolved, so swapping two handles’ definitions cannot preserve the root. The initiating agent proposes a context set by listing the required pattern identifiers; the receiving agent independently resolves each identifier, constructs the same Merkle Tree, and proceeds only if both the declared scheme and roots match. The agents then exchange the Merkle Root of their respective trees:

$$R_{context} = \text{CatalogMerkleRoot}(\{(h_1, \mathcal{H}(d_1)), (h_2, \mathcal{H}(d_2)), \dots, (h_n, \mathcal{H}(d_n))\}) \quad (5)$$

If the full roots and schemes match, agents have a collision-resistant commitment that every selected handle resolved to the same canonical definition—and therefore share an identical external semantic contract for each—allowing them to proceed with optimized, short references. If the roots or schemes differ, the system detects divergence and halts immediately. This ensures that *compliant* agents never proceed with subtly different baseline definitions of the instructions, effectively detecting divergence before coordination begins (Figure 3). The comparison must travel through an authenticated channel if an active attacker is in scope. We note that this is an enforcement mechanism, not a Byzantine guarantee: it assumes agents voluntarily participate in the handshake. A non-compliant agent that bypasses verification entirely is outside the protocol’s threat model, analogous to how TLS guarantees confidentiality only for parties that complete the handshake.

The current convenience tools `sema_propose_context` and `sema_verify_context` exchange an eight-hex-character catalog-root prefix and label it explicitly as cooperative drift detection, not adversarial security; they require the declared scheme even in this mode. Deployments requiring collision-resistant identity assurance compare the full catalog root under that scheme through an authenticated channel. A full semantic-set root answers the weaker question of whether the raw definition-digest sets match; under canonicalization v2, those digests can still change when target handles in structured references are renamed.

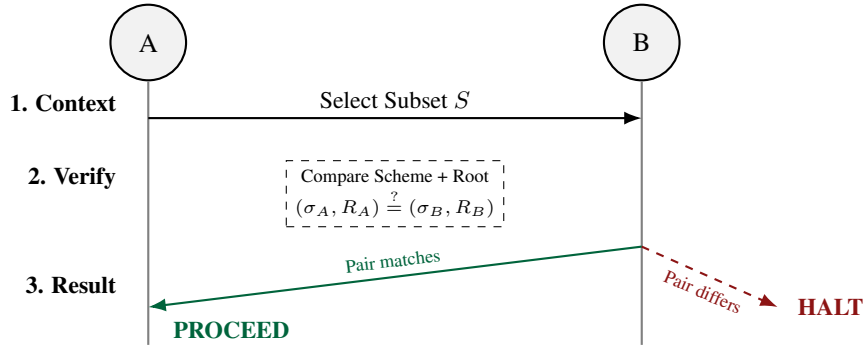


Figure 3: Fail-closed handshake. Agents compare the root scheme and Merkle Root of the agreed pattern subset. Divergence in either triggers immediate halt.

This represents a deliberate inversion of Postel’s Law [?]. Where the Internet’s robustness principle advises “be liberal in what you accept,” Sema inverts the receiving side: be strictly conservative in what you accept, and reject what you cannot verify. This paradigm shift prioritizes safety over interoperability: silent failures stemming from unverified vocabulary assumptions are systematically prevented because ambiguity triggers explicit rejection rather than best-effort interpretation. In multi-agent systems where misalignment can cascade catastrophically, fail-closed semantics are not merely preferable; they are essential.

The scope of this guarantee deserves explicit statement. Sema’s fail-closed behavior is a property of *compliant deployments*, not an unconditional property of the protocol: an agent that never calls `sema_handshake` can coordinate

on any definition it wishes. This is analogous to how TLS guarantees confidentiality only for parties that complete the handshake, not for parties that choose plaintext. What Sema provides is (a) a guarantee within cooperative deployments that divergent meaning cannot hide behind shared surface labels, and (b) a structural boundary in adversarial deployments: a non-compliant agent is immediately visible as non-compliant, because any coordination protocol that requires handshake tokens will reject its messages. Enforcement at the coordination-protocol level—making handshake calls mandatory in the agent’s execution harness rather than merely available—is an integration decision beyond the scope of the Sema protocol itself, and is discussed in Section 9.

Proposition 1 (Canonical Hash Collision Bound). *For a vocabulary of size $|\mathcal{V}|$ with definitions of average entropy H , the probability of two distinct canonical definitions producing the same hash is bounded by:*

$$P(\text{collision}) \leq \frac{|\mathcal{V}|^2}{2^{257}} \approx 0 \quad (6)$$

This follows directly from the collision resistance of SHA-256. With $|\mathcal{V}| = 456$, collision probability is negligible ($< 10^{-72}$).

Proposition 2 (Fail-Closed Detection). *If agents A_i and A_j have definitions $d_i \neq d_j$ for pattern P , then $\mathcal{H}(d_i) \neq \mathcal{H}(d_j)$ with overwhelming probability, triggering protocol halt for compliant agents.*

Proposition 3 (External Contract Identity). *If agents A_i and A_j both resolve identifier h against an honest content-addressed store and obtain the same canonical definition d with $\mathcal{H}(d) = h$, then they hold an identical external semantic contract for h , independent of their private internal semantic models M_i, M_j . Sema does not prove $M_i(h) = M_j(h)$; it proves that the externally resolved object is the same, so any divergence in downstream behavior must be attributable to the agents’ interpretations of a shared definition rather than to the agents holding different definitions.*

Proposition 4 (Compression with Resolvable Expansion). *Let $|r|$ be reference length (handle + 4-char hash ≈ 15 characters) and $|d|$ be definition length. Compression ratio is $|d|/|r|$. Expansion against the referenced canonical object is deterministic: given access to d , $\text{expand}(r) = d$ is a pure function returning identical bytes on every invocation. Deployments that need cryptographic assurance verify the expansion against the full pattern hash or an agreed context/vocabulary root; the short stub is the readable wire token, not the complete security boundary.*

3.5 The Type System

This subsection describes the type system of the Sema Bootstrap Library. It is a *particular* type system, not *the* type system: the protocol above only requires an explicit schema whose canonical form can be reproducibly hashed. Different communities or domains can therefore build different Sema dialects while sharing the same cryptographic carrier. Those dialects may not interoperate field-for-field, but their boundaries are explicit: hashes still identify canonical objects, and handshakes still expose divergence. In this sense Sema is closer to Unicode than to a universal language: it standardizes the carrier, not every vocabulary spoken through it.

We chose the agent-protocol schema below because structural typing lets the same verb operate across domains as long as subjects satisfy a common contract, and polymorphic dispatch lets an agent request “any pattern that implements Deep(Discover)” rather than pinning a specific hash. Readers interested only in protocol-level guarantees may skip to Section 4; readers interested in how Sema handles polymorphism, composition, and wiring across 456 patterns should continue here.

A central tension in vocabulary design is determining when a change in attributes constitutes a change in identity. The bootstrap library adopts a structural resolution: qualitative differences create distinct patterns, while quantitative differences create parameters with hashed range contracts. For example, `ExponentialBackoff#f5e5` with a `base_delay` of 100ms and the same pattern with a 5-second delay share the same semantic identity because the hash captures the parameter range contract, not the instance value. However, if changing a variable alters the failure mode, such as switching from busy-waiting to database storage, it becomes a distinct pattern (e.g., `SpinLock` vs. `Lease`). This allows the runtime to validate instantiation values against the hashed range contract while preserving the conceptual unity of the pattern.

To enable infinite reuse without recreating the API chaos of traditional systems, Sema employs Typed Interfaces rather than named fields. The `dependencies` object is strictly partitioned into four mutually exclusive categories, creating a directed acyclic graph (DAG): `accepts` defines passive input data patterns consumed by the mechanism; `yields` defines passive output data patterns produced by the mechanism; `composes_with` defines active tool or logic patterns explicitly invoked by the mechanism; and `references` defines patterns used for conceptual clarity but not execution. This partitioning enforces a “Single Source of Truth” where every semantic link lives in exactly one place, and decouples verbs from specific noun instances. A `LogisticsAgent` locking a shipping container and a `MedicalDrone` locking a patient record both invoke the same pattern because both subjects satisfy the `accepts` interface (e.g., `"accepts": {`

"subject": "sema:UniqueHandle..." }). Variable names are irrelevant; type compatibility is everything. This architecture ensures that the 456 core patterns can be composed into infinite variations, as agents from different domains can coordinate verbs simply by sharing common noun definitions.

The integrity of these compositions is enforced by the Explicit Wiring Rule, which welds the abstract type to the concrete logic. The compiler mandates that every element in a pattern’s signature (the claim, e.g., `Check(Safety)`) must be explicitly imported in the dependencies and invoked in the pattern’s text fields (mechanism, invariants, preconditions, postconditions, or failure modes) via a template key. This renders “Phantom Signatures” impossible to pass through the Sema Compiler; an agent cannot claim to implement a capability unless it cryptographically links to the specific pattern in its mechanism.

The Sema Compiler resolves these high-level intents into executable low-level handles. When an agent executes a polymorphic intent such as `Deep(Discover)`, the compiler queries the vocabulary graph for a pattern declaring that signature (e.g., `DeepResearch#f72c`) and resolves the dependency chain. This compilation step decouples intent from implementation, allowing the system to upgrade the underlying algorithms, such as swapping a heuristic check for a rigorous crypto-audit, without altering the agent’s cognitive script.

Table 1: The Deep Registry: Resolving Polymorphic Rigor. Fast and Slow refer to lightweight versus heavyweight resolution strategies.

Intent (Query)	Source Pattern (Fast)	Resolved Target (Slow)
Deep(Heuristic)	HeuristicSnap#f7cc	MentalSim#6de8
Deep(Trace)	Trace#7af1	GenealogicalTrace#4945
Deep(Discover)	Discover#f6e0	DeepResearch#f72c
Deep(Nature)	Nature#9ac1	LivedProof#9c65

This design yields the property that distinguishes Sema from all prior content-addressing systems: **the hash functions as a word in the natural-language stream that agents exchange**. A precision is warranted here. We do not claim the LLM reasons directly over hash bytes; the local runtime hydrates the full definition into the system prompt before inference, so the cognitive substrate remains the unpacked mechanism text:

$$\text{Prompt} = \text{System} \oplus \text{Hydrate}(\text{Patterns}) \oplus \text{Query} \quad (7)$$

What is novel is that the *wire-level token* is simultaneously a readable handle (`PropheticQuorum#baab`) and a pointer into a cryptographically verifiable definition space: the hydration step is itself content-addressed (different bytes on the wire $\rightarrow$ different lookup $\rightarrow$ detectably different definition injected), so any divergence between what the sender thought they said and what the receiver thought they heard surfaces as a hash mismatch rather than as a silent misinterpretation. Unlike Git hashes (infrastructure plumbing invisible to developers), IPFS hashes (file addresses), or SDH URIs (formal graph references), Sema identifiers are words in the medium communication already uses—each carrying integrity that is syntactic for the reader (the 4-char stub reads fluently) and cryptographic for the runtime (hydration is checked against full hashes or agreed roots). There is no serialization boundary between communication and verification. We accept the token tax in the context window to gain that safety on the wire.

4 The Bootstrap Library

This section presents the Sema Bootstrap Library: the Schelling-point vocabulary introduced in Section 3. The library is not a closed canon but an initial seed; if an agent requires a different locking mechanic, it can permissionlessly mint `StateLock_v2` without breaking the protocol.

4.1 Methodology

We emphasize that the initial vocabulary is a pragmatic bootstrap, not the output of a principled selection process. A different author would produce a different vocabulary using the same protocol, and that is by design: the protocol’s value is independent of any particular library. The 456 patterns emerged iteratively from four sources: distributed systems primitives, the cognitive-decomposition architecture developed in [?], LLM-assisted gap-finding, and the author’s own domain intuitions about agent failure modes. Creativity protocols such as Adversarial Evolutionary Generation [?] were used for some frontier patterns.

The process became more structured as the library grew. Refinement checkpoints used a recurring “Taxonomist” role to reject duplicates, merge overlapping concepts, and enforce the Noun/Verb separation; repeated failures became rules

and guidelines (Section 4.8); and the database layer enforces DAG acyclicity, dependency-hash transitivity, bidirectional wiring checks, and Rule-G layer direction. The atomic `sema apply` step operationalizes these constraints by validating additions and removals before mutation, catching handle-format errors, signature mistakes, layer-direction violations, near-duplicates, and rigor warnings on every mint and CI pass.

Minting is gated by two criteria. A concept earns a pattern if it requires *protocol consistency*—multiple agents must coordinate on the exact semantics, as with `StateLock#c9c2`—or if specifying it produces *structured thinking*, forcing a loosely used English concept into an invariant-bearing mechanism. English suffices when neither criterion is met. Pattern *generality* is then checked by a broad-use test: the mechanism and invariants must capture what every legitimate deployment context needs, while context-specific features become descendant territory via `extends` (e.g., `Lock#95c2` $\rightarrow$ `Mutex#8dd8`, or `Task#a850` $\rightarrow$ `BoundedTask#fa24`). The test prevents both overfit mechanisms that break on legitimate variants and toothless abstractions that constrain nothing.

After the refinement pass described in Section 4.8, the library has no duplicate mechanisms or semantic collisions detected by the applied audits (hash collision, canonical-form equality, and embedding-similarity sweeps above a threshold) across 456 patterns. These audits and the Taxonomist role maintain quality, but they do not guarantee functional uniqueness (see §9, “The Synonymy Limit”). We make no claim to completeness or optimality.

4.2 The Civilization Stack

The vocabulary is not a flat list; it is structured into four fundamental layers mimicking a civilization stack. At the base lies **Physics**, substrate primitives that obtain regardless of any author: thermodynamics of coordination that cannot be “wished away” (e.g., `Mutex#8dd8`, `Entropy#d5f7`, and recently-added substrate concepts `Gradient#ecfc`, `Equilibrium#4f03`, `Conservation#0b32`, `Distance#dfdf`, `PhaseTransition#b775`, `Attractor#0036`, `MutualInformation#4036`, and `Measurement#511d`). Above this rests **Mind**, mechanisms that structurally require cognition (e.g., `BayesUpdate#1641`), where judgment must be exercised and a single isolated agent can execute the mechanism. Emerging from these is **Society**, mechanisms that structurally require ≥ 2 independent parties with potentially divergent state (e.g., `Consensus#5402`). Enclosing all three is **Infrastructure** (Figure 4): authored structures and operations that do not require cognition to execute—data types, composite topologies, and mechanical operations (e.g., `ComputeBudget#4091`).

The placement of each pattern is governed by the *mechanism-sufficiency test*: what does the pattern’s mechanism structurally require to execute? A pattern whose mechanism requires nothing more than substrate goes to Physics; one that requires authored structure but no cognition goes to Infrastructure; one that requires cognition but can be executed by a single isolated agent goes to Mind; one that structurally requires another party with separate state goes to Society. The axis is deliberately *not* what the pattern is typically used for or what it conceptually operates on. This test is the governing principle recorded in `docs/core/philosophy.md` §3.1.

The four-layer ordering should be read as a design discipline for this bootstrap library, not as a claim that all semantic vocabularies must decompose into these layers. The motivation is operational rather than ontological: lower layers should not depend on higher layers because doing so makes mechanical guarantees depend on social or cognitive assumptions. Resource and execution guarantees should not depend on reasoning strategies or social protocols, while social protocols may safely depend on lower-level execution and reasoning primitives. The “Semantic Inversion Attack” analyzed in §7.5 is the concrete failure mode this discipline prevents.

This stratification enforces a property similar to Stewart Brand’s “Pace Layering” [?]: lower layers provide stability for the turbulence of upper layers. By rigorously separating **Physics** (immutable rules like `Entropy#d5f7`) from **Society** (mutable agreements like `Constitution#33f6`), the system achieves resilience. Agents can rewrite their social contracts or voting protocols without breaking the fundamental laws of time and causality that underpin their reality. In flat ontologies, a change in governance often breaks the definition of truth; in Sema, the laws of physics are immune to political amendment.

To prevent semantic inversions, the dependency graph must form a DAG—the hard constraint enforced by the Merkle hashing mechanism (a cycle would create infinite hash recursion). Layer direction is a second constraint with a more subtle history. Layer metadata is deliberately excluded from the hash, so the protocol is agnostic to layer choice: a pattern’s identity is fixed by its mechanism and dependencies, not by which tier a library chooses to file it under. This is a real degree of freedom—originally we treated layer direction as a style guide rather than a law, on the grounds that the protocol would function identically whether a particular concept was filed one level up or down, and a library can legitimately adopt a weaker policy and still interoperate at the hash level with one that enforces layer direction strictly.

The bootstrap library chose to tighten the rule through refinement (Section 4.8): hard dependencies (`accepts`, `composes_with`) must now flow from higher layers to lower (Society $\rightarrow$ Mind $\rightarrow$ Infrastructure $\rightarrow$ Physics), enforced

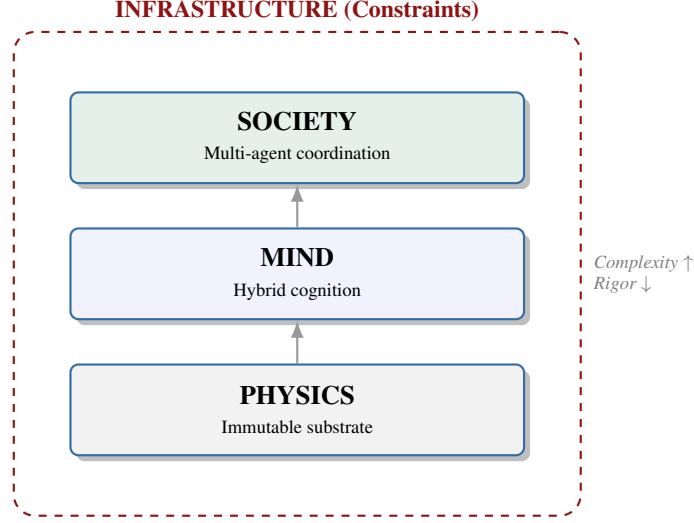


Figure 4: The Civilization Stack. Infrastructure constrains the stack; Physics supports Mind, which enables Society. Arrows indicate dependency: upper layers rely on guarantees from lower layers.

as a *sema* apply gate, while soft citations (*references*) and outputs (*yields*) are exempt and allow semantically necessary cross-layer links (for example, *Belief#9ef7* in Infrastructure soft-references *Agent#6087* in Mind because beliefs are conceptually held by agents, and the reference is acyclic). The current vocabulary contains 456 patterns organized into 12 categories across these four layers (Table 2; complete taxonomy in Appendix B), with zero patterns violating the layer-direction rule at this version.

4.3 The Grammar of Agency

The vocabulary is not a flat list of tools but a compositional grammar that enables agents to construct novel intents from atomic units. Ring 0 *Primitives* act as the fundamental building blocks of agent cognition. Five foundational verbs map to dimensions of autonomous inquiry: *Discover#f6e0* (Breadth) acts as the Scout, querying the horizon to find new nodes, agents, or paths; *Deep#16b0* (Depth) serves as the Scientist, escalating from heuristic proxies to rigorous implementations; *Trace#7af1* (Time) functions as the Historian, reconstructing causal chains and ensuring provenance; *Check#6890* (Validity) operates as the Critic, filtering inputs against invariants; and *Stigmergy#a0f0* (Space) works as the Builder, writing persistent signals for indirect coordination. Alongside these verbs, noun primitives such as *Belief#9ef7*, *Agent#6087*, and *Constraint#7013* provide the objects of agency.

These primitives enable a higher-order grammar where complex behaviors compose from atomic units: *Check#6890*(Condition) evaluates a target *Condition#b480* and enforces a fail-closed halt if False; *Trace#7af1*(Target) wraps a process to generate an immutable lineage log; and *Stigmergy#a0f0*(Signal) marks the shared environment with a signal for others to discover.

Ring 2 *Macros* are executable patterns composed of primitives via semantic import. (Ring 0–2 describes compositional complexity; Tier 1–3, Section 4.5, describes safety rigor. Both are stored in the unhashed metadata overlay.) For example, *AgentDiscover#4c06* imports *{{Discover#f6e0}}* acting on *{{Agent#6087}}*, while *TraceBelief#639c* imports *{{Trace#7af1}}* acting on *{{Belief#9ef7}}*. This grammar allows agents to express intent via queries like “I need to Deep(Trace) this belief,” which the runtime resolves to concrete executable patterns.

This distinction reveals a bicameral architecture. Primitives serve as the query interface, allowing agents to identify logical gaps in the vocabulary. Macros like *IdentityHandshake#8653* serve as the social interface, enabling efficient coordination through compression. Instead of negotiating four separate primitives in four round-trips, agents verify a single Macro hash in one. Primitives are for querying; Macros are for talking.

The verb/noun distinction introduced in the Type System (Section 3.5) also addresses the “schema drift” problem where implicit schemas lead to subtle incompatibilities. Sema formalizes the “shape of work” into content-addressed Data Patterns. *Task#a850* represents a recursive definition of intent that inherits constraints from its parent and explicitly defines its *AcceptSpec*. *Solution#6bd0* acts as a container for work product, encapsulating provenance and a component tree for supply-chain auditing. *UniqueHandle#1853* serves as a rivalrous resource pointer obeying linear

Table 2: Sema vocabulary: 456 patterns across 12 category names and 4 layers

Layer	Category	N	Examples
Physics	Primitives	16	Attractor, Causation, Conservation
	Time	1	CausalBarrier
Infrastructure	Data Structures	96	AcceptSpec, Aesthetics, Anomaly
	Primitives	52	Act, Actor, Aggregate
	Verification	9	AuditTrail, Compatibility-Check, ExplainBeacon
Mind	Inference	22	BaseRateInclude, BayesUpdate, BreadthGovernor
	Memory	15	BeliefTracking, ChunkMerge, ContextCompress
	Reasoning	61	Abduction, BackwardChain, Bisect
	Strategy	81	AdversarialSteel, Agent, AnalogyBridge
Society	Coordination	12	Compromise, Consensus, ConsensusFinder
	Economics	9	AtomicBid, AttentionMarkets, Award
	Governance	7	AnchorDrop, Constitution, DocumentedOverride
	Protocols	75	AdversarialProof, AgentDiscover, AgentProtocol
Total		456	

logic to prevent double-spending of physical assets; if Agent A transfers the handle to Agent B, Agent A loses access immediately. To ensure structural compatibility, Noun definitions may include a validation schema in the `data_schema` field (e.g., `Task#a850`, `Bid#f71c`), embedding constraints directly into the identity unlike external standards such as SHACL [?]. This allows agents to construct rigorous semantic sentences:

“I Delegate#5bfe this Task#a850 to you, expecting a Solution#6bd0 that satisfies this AcceptSpec#a724.”

4.4 Cognitive Architecture: The Solver Stack

The vocabulary encodes a complete cognitive architecture whose theoretical foundations, including the Universal Solver Tree, polymorphic solver nodes, and the marginal-value rule governing decomposition depth, are developed in a companion preprint on fractal intelligence [?]. Sema provides the content-addressed implementation layer where each concept from the Composable Reasoning Protocol becomes a machine-verifiable pattern.

The architecture is anchored by the `PolymorphicSolver#a3ff`, which implements the five-surface Solver Contract across any substrate (LLM, human, hybrid, tool-using agent, or nested composition) and acts as the default implementer within the `UniversalSolverTree#3708`. The abstract contract itself is `Solver#` — an interface any `Agent#` can take on for the duration of a `Task`. Each Solver node presents a uniform interface but dynamically decides whether to execute the work directly or to decompose via `ConceptualDecomposition#1ea3` (distinct from generic `Decompose#db21` by its contract-bound requirement: each sub-concept exposes the Solver interface and is therefore delegatable) and delegate to child Solvers. The five surfaces: `Manifest` (`Card#39f7`), `Execute` (`PolymorphicSolver#a3ff` itself — the universal atom), `Consult` (`SocraticLoop#164a`), `Verify` (`Validate#bd94`), and `Feedback` (`PerformanceSignal#be83`). Inputs are encapsulated in a `Task#a850` with full constraint inheritance, while outputs are returned as a `Solution#6bd0` with full provenance. The `ComputeBudget#4091` acts as a pre-execution gate, and `MarginalValueRule#60d2` governs whether further decomposition is worth the cost. The claim that reasoning extends beyond the capacity of any individual model through typed composition is developed in [?].

4.5 Contract Formalism Tiers

Patterns vary in formal rigor. We classify them into three tiers:

Tier 1 (Ironclad) comprises 316 patterns with formal contracts (invariants, preconditions, postconditions), making them safe for autonomous execution without trust assumptions.

Tier 2 (Honesty-Dependent) includes patterns with clear mechanisms that assume alignment-seeking agents. These are safe for cooperative contexts, but adversarial contexts require Tier 1 alternatives due to collusion risks (e.g., `SteelmanCheck#a375`).

Tier 3 (Experimental) contains patterns with novel concepts requiring further refinement, serving as seeds for vocabulary expansion.

Adversarial stress testing (Section 7.5) can cause tier downgrades: a pattern initially classified Tier 1 may be demoted to Tier 2 upon discovery of exploitable trust assumptions.

4.6 The Babbage Principle of Cognition

The vocabulary operationalizes the *Babbage Principle* [?] for artificial intelligence: the division of cognitive labor allows each sub-task to be routed to the cheapest competent solver. Rather than employing a high-cost generalist model for every reasoning step, the `FractalIntelligence#04a6` architecture uses `Decompose#db21` to break problems into sub-tasks that can be handled by specialized, lower-cost agents or deterministic scripts. Recent empirical work on atomic “micro-agent” decomposition [?] and DeepMind’s AlphaEvolve [?] confirm the economic necessity of this approach. See [?] for the full treatment.

4.7 Example Patterns

Sema’s utility is best understood through concrete examples from the Tier 1 library. `StateLock#c9c2` (Time) provides atomic coordination via temporary state fusion, where two agents fuse a subset of writable state such that changes require both signatures. The lock auto-dissolves on timeout, though a failure mode exists where deadlock can occur if one agent disappears. `SpectralTune#c0aa` (Protocols) verifies ontology alignment before data transfer; the sender transmits hash-based tuning signals and the receiver proves matching context, preventing mismatch but risking infinite loops if ontologies are slightly different.

For cognitive safety, `SteelmanCheck#a375` (Reasoning) mandates the generation of the strongest counter-argument, forcing the instantiation of a “Critic Persona” that attacks the agent’s own plan; however, this is vulnerable to sycophantic critics that fail to genuinely challenge. Finally, `WhyClimb#911f` (Reasoning) enables recursive problem abstraction via iterative “Why is this a problem?” ascent. The agent climbs the abstraction hierarchy until reaching the “Ceiling,” where the problem is still actionable but the solution space is maximized, though care must be taken to avoid the failure mode of climbing too high, such as attempting to solve “Entropy” instead of “Fix Bug.” Full specifications for these patterns appear in Appendix A.

4.8 Refinement

The vocabulary is not a static artifact; it evolves through new mints, retirements, mechanism rewrites, and relocations. Refinement is one concrete example of how that evolution proceeds: a cross-pattern pass in which the library is swept end-to-end against the current governing rules, rather than pattern-by-pattern at mint time. With hundreds of patterns in place, questions of layer coherence, redundancy, and principle consistency cannot be answered one pattern at a time; they require systematic passes against shared formal criteria. Refinement is the stage of the methodology where those passes happen.

A refinement pass is governed by the versioned vocabulary design manual rather than by ad hoc judgment. Proposed relocations, retirements, mechanism rewrites, and rule clarifications are tested against the current manual, adversarially reviewed, and finally adjudicated by a human maintainer. The paper-level claim is the audit discipline, not a particular review workflow: each accepted change must state the rule it depends on, the failure mode it prevents, and why the alternative was rejected.

Concrete refinement decisions illustrate the character of the pass. `Bid#f71c` was initially filed under Society/Economics on the intuition that bids are a social-economic concept; the mechanism-sufficiency test reassessed this kind of judgment call by asking what the pattern’s mechanism structurally requires (in the Bid case, the decision retained the Society classification because the mechanism requires a counterparty, but the same method resolved many analogous ambiguities

in the opposite direction). Earlier drafts also left layer direction unenforced on the argument that layer metadata is not hashed and the protocol is therefore agnostic to layer choice; the refinement kept that degree of freedom intact at the protocol level but tightened the bootstrap library’s policy to enforce layer direction as a `sema` apply gate. Many other decisions were similar in character: a concept that had been filed by vibe or typical-use-case was re-tested against what its mechanism structurally required, and moved when the two disagreed.

A second class of refinement decision concerns *scope* rather than *placement*: a pattern may be coherent, well-mechanized, and correctly layered, yet still inappropriate for the default library that downstream consumers install by default. During refinement, a subset of patterns whose canonical applications involve capability amplification, social manipulation, evasion, or cryptoeconomic binding (for example, `AmendLaws`, `ChaosDrift`, `CryptoShred`, `IdentityMask`, `MirrorStake`) were judged to belong in a separate experimental shelf rather than in the default library. The patterns remain resolvable at the hash level—both databases stay in the repository—but the default install stays conservative, and engaging the experimental shelf is a deliberate act by a user who has read what they are opting into. This splitting decision is made at refinement time, per-pattern, on the basis of how the mechanism composes with realistic deployment contexts rather than on whether the mechanism is well-formed (the latter is already checked by the other gates). The resulting distribution architecture is described in Section 4.9.

The result is relevant to the paper’s main claim: a content-addressed vocabulary is only as rigorous as the process that produces its definitions. The per-pattern reasoning—each pattern’s intended use, the rationale behind its invariants, and the design commentary supporting relocations, rewrites, and retirements—is shipped with the library as a per-release design manual,³ so that the review surface of the vocabulary is visible rather than implicit in commit history.

4.9 Distribution: A Canonical Vocabulary that Consumers Pull

The content-addressed model enables a distribution architecture where a single canonical vocabulary evolves centrally while every consumer maintains its own local database. The canonical vocabulary is split along the safety boundary established in Section 4.8: a default library of broadly applicable patterns, and a separate experimental shelf holding higher-risk patterns. Both databases stay in the repository and all hashes remain resolvable, but the default install stays conservative.

Each consumer holds its own copy of the vocabulary, which is not a redundant mirror but a purposeful locus: the local database lets the consumer deviate from upstream (by deleting a pattern, pinning an older version, or authoring local extensions) without those deviations bleeding into every other consumer. Consumers stay aligned with the canonical vocabulary through a *pull* operation that synchronizes along the dependency graph, so a pattern is integrated only after its dependencies are already verified in the local database. Retirements carry successor metadata, so a reference to a retired handle redirects to its replacement rather than dangling; renames carry the same, so local code that cites an old handle continues to resolve. The local database is always self-describing: it knows its vocabulary root, the hashes it carries, and the upstream patterns it has explicitly declined.

The workflow the system enables is that the canonical vocabulary evolves in place. When refinement sharpens a mechanism, mints a new Physics primitive, or retires a drifted concept, the change lands in the canonical database and is reachable by any downstream consumer at the next pull. No consumer has to track changes manually; equally, no consumer is forced to accept a change they object to. The content-addressed guarantee makes this trustworthy: a pulled pattern’s bytes are exactly the author’s bytes, and if a consumer’s local database diverges from upstream the divergence is explicit and auditable at the hash level. The canonical library therefore behaves less like a fixed standard and more like a living commons: definitions sharpen over time, safety boundaries stay explicit, and every consumer’s vocabulary is traceable to a specific point in that evolution.

The centralized-canonical model is one governance path among several the protocol supports, not a constraint the protocol imposes. Fully decentralized variants are compatible with the same hashing and Merkle machinery: multiple coexisting canonical libraries (each with its own root and its own maintainer community), federated governance where pattern updates require a quorum of signing libraries before they propagate, fork-and-merge workflows where any consumer can become a fork point that others follow, or fully peer-to-peer minting where no canonical source exists at all. The content-addressed guarantee holds across all of them: a pattern is defined by its bytes, not by who hosts it, so two consumers referencing the same hash reference the same canonical definition regardless of which governance structure produced it. We chose a canonical-pull model for the bootstrap because it minimizes onboarding friction and gives the refinement methodology a clear locus, but the decentralization requirement identified in the earlier Living Taxonomy work [?] is compatible with stronger choices, and we do not claim the current model is the endpoint.

³<https://github.com/emergent-wisdom/sema/blob/main/docs/manuals/vocabulary-design.md>

5 The Living Taxonomy of Thought (Dynamics)

The static properties described so far (hashing, types, pattern cards) exist to support a dynamic conceptual layer in which agents can reuse prior distinctions, identify meaningful departures from them, and leave those departures behind as new cognitive infrastructure. This architecture builds on the concept of a “Living Taxonomy of Thought” sketched in earlier work [?] as a component for self-improving AI. The core protocol described in §3 is implemented today; the Living Taxonomy and the Experience Matrix sketched in this section are the architectural consequences the protocol makes possible—they are what content-addressed meaning unlocks once the foundation is in place. We separate carefully what is *implemented* (hashed specialization claims through `extends`; unhashed replacement records through `_meta.supersedes`; Semantic Telepathy at the pattern-identity level; and the mint, validation, and hash-computation infrastructure) from what is *proposed* (general version-lineage traversal and analytics; the automated Genesis Loop; the Judge primitive’s scoring implementation; Contextual and Experiential Telepathy). The proposed components are described here because they are the targets the implemented infrastructure was designed to support; Section 9 catalogs their current status explicitly.

5.1 The Creative Discovery Loop

The core engine of the Living Taxonomy is the *Genesis Loop*, a recursive process where the AI authors its own cognitive infrastructure. Unlike traditional learning (which adds facts), this process upgrades the machinery of thought itself, aligning with recent frameworks for LLM Self-Evolution [?] which identify iterative refinement and self-correction as key drivers for next-generation intelligence. This architecture also mirrors the *Nested Learning* paradigm [?], where rapid, inner-loop adaptation (local usage) is distinct from but eventually consolidated into stable, outer-loop weights (the global hash), preventing catastrophic forgetting of the semantic structure.

Both Sema and earlier work on ontological discovery [?] share a key structural insight: forcing concepts into a categorized space reveals structure that ad-hoc naming obscures. In that work, a dual-agent system imposed taxonomic categories on an open-ended conceptual domain; here, the Sema layer hierarchy and typed pattern cards impose analogous structure on coordination primitives. The contribution of Sema is not to make taxonomy placement part of identity—metadata remains unhashed—but to make the specification produced by that categorization content-addressed. If categorization pressure changes a pattern’s mechanism, invariants, or dependencies, the hash changes; if only the library filing changes, identity remains stable.

The Genesis Loop is a proposed protocol for vocabulary self-evolution; the current implementation provides the infrastructure it would require (pattern validation, hash computation, dependency resolution) but does not automate the loop itself.

The loop would proceed in four phases. First, in *Hypothesize (Generativity)*, an agent detects friction or explores latent space (via `ConceptBlend#c9f1`) to generate a candidate mechanism. Second, during *Judge (Evaluative Merit)*, the candidate would be passed to the `Judge#6906` primitive for a scalar assessment of structural merit. Third, the *Harden (Adversarial Evolution)* phase would subject the pattern to `AdversarialSteel#52bc`, where a “Devil’s Advocate” attempts to exploit its invariants. Finally, in *Mint (Crystallization)*, the survivor is hashed, becoming an immutable pattern available for immediate citation and reuse. The infrastructure for the final step—validation, hashing, and atomic minting via the `sema_mint` tool—is fully implemented; the preceding three steps require either human judgment or LLM-based evaluation that remains future work.

The Hypothesize phase can draw candidates from several sources: observed coordination failures, systematic refinement passes, imported domain mechanisms, or lexical exploration. These sources differ in where the candidate comes from, but not in how it becomes part of the library. Each must pass through the same downstream gates—Judge, Harden, Mint—so diverse ideation converges into a single, uniformly vetted vocabulary.

5.2 Evaluation as a Primitive: The Judge

To enable the Genesis Loop, we introduce a new Ring 0 primitive: `Judge#6906`.

While `Check#6890` queries *Validity* (True/False), `Judge#6906` is specified to query *Merit* (0.0 to 1.0). Unlike probabilistic claim analysis systems that rely on model confidence [?, ?], the proposed Judge would evaluate structural properties of the pattern (connectivity, orthogonality) rather than its truth value. It is intended to operationalize the “metacognitive training” that distinguishes deep insight from sophisticated rhetoric.

$$\text{Judge}(\text{Pattern}) \rightarrow \text{Score} \in [0, 1] \quad (8)$$

This primitive allows the taxonomy to self-prune. Agents configure their uptake thresholds based on these scores, ensuring that high-merit patterns propagate while low-merit noise dies in the mempool.

5.3 Phylogeny: Specialization and Version Lineage

A living taxonomy needs to represent two different relationships. The hashed `extends` field records specialization: a child claims to be a narrower kind of one exact, immutable parent definition, and the graph exposes that claim as an `IS_A` edge. The unhashed `_meta.supersedes` list records version lineage: the current card replaces one or more earlier identifiers. The first relation is part of what a card means; the second records how that meaning changed over time.

```

1 {
2   "handle": "BoundedTask",
3   "extends": "sema:Task#mh:SHA-256:
      a85010f508f85055170c396c8fa390125c1206888c565ac10651f3969a339f8d",
4   "mechanism": "A specialized Task enforcing budget...",
5   "dependencies": { ... },
6   "_meta": {
7     "supersedes": ["sema:BoundedTask#mh:SHA-256:..."]
8   }
9 }
```

The implementation stores and queries the specialization graph, while `supersedes` is currently consumed by the pull reconciliation path. A general query such as “show the evolutionary path through every published version,” or analytics identifying stagnant and rapidly speciating lineages, requires a dedicated traversal over the replacement records and remains future work.

5.4 The Experience Matrix: From Language to Telepathy

Experiential Telepathy remains a theoretical proposal. The current implementation supports Semantic Telepathy (Level 1: agents share meaning through content-addressed patterns) and provides the schema infrastructure for higher levels, but tensor serialization and gradient transfer are not implemented.

Content-addressing applies the same principle at three levels of granularity, each enabling a different form of “telepathy” between agents:

Semantic Telepathy (implemented) Agents share individual pattern hashes. When two agents reference the same `sema_id` and resolve it against any honest content-addressed store, they are guaranteed to share the same definition. This is the foundation: telepathy of *meaning*.

Contextual Telepathy Agents hash their full conversational state—the accumulated context window at a checkpoint—producing a content-addressed snapshot of shared understanding. A third agent joining a swarm mid-session can verify “I hold the same context you held at step 12” without replaying the full transcript. This enables resumption, forking, and trust verification across agent handoffs: telepathy of *shared understanding*.

Experiential Telepathy Agents serialize their learned state (e.g., Q-tables, LoRA adapters, or success-probability tensors) into content-addressed *Matrix Patterns* defined by `ExperienceSharding#d1c3`. Unlike a text definition which explains how to do a task, a Matrix Pattern transfers the gradients of having done it. If Agent A optimizes a specific `ExploreExploit#d570` strategy after 10,000 trials, it mints a heuristic matrix. Agent B imports this matrix, linearly interpolating it with its own priors to instantly acquire Agent A’s intuition: telepathy of *acquired skill*. This echoes the *Nested Learning* paradigm [?], where rapid inner-loop adaptation is distinct from stable outer-loop consolidation. (Matrix Patterns are not yet implemented; see Section 9 for status.)

Each level builds on the one below: you cannot share context without agreeing on terms, and you cannot share experience without agreeing on context. The result is a ladder from language to telepathy—the mathematical transfer of understanding without the bottleneck of verbal explanation.

6 Implementation

The Sema system is implemented as an open-source Python package organized around five core components. Specific line counts, tool inventories, and graph-store type catalogues are intentionally omitted here; they drift across releases and are documented in the repository.⁴

⁴<https://github.com/emergent-wisdom/sema>

The implementation has three boundaries readers should distinguish before §6’s evaluation. *Implemented and enforced*: canonicalization, hashing, schema validation, atomic mutation, dependency graph constraints, and the MCP handshake. *Implemented but optional*: hybrid search, embedding navigation, dependency expansion, browser tooling, pull-mode synchronization, and the Taxonomist refinement role—these run as agent or operator workflows but are not protocol-level requirements. *Specified but not enforced at runtime*: the Judge#6906 primitive’s scoring, the Genesis Loop, and the Experience Matrix—these are described in §5 as the targets the implemented infrastructure was designed to support, with their current status catalogued in §9.

Schema Validation. Every pattern passes through a Pydantic validation pipeline with 14 custom validators enforcing: CamelCase handle format, signature syntax (`Verb(Noun)`), parameter completeness, category-layer alignment against the taxonomy tree, bidirectional dependency checking (all `{{placeholder}}` references must appear in the dependency map, and all declared dependencies must be referenced in the pattern’s text fields—mechanism, invariants, preconditions, postconditions, or failure modes), and the Explicit Wiring Rule (all signature types must appear in dependencies). A pattern that fails any validator is rejected before hashing. The enumeration of checks and how they accumulated over time is covered in Section 4; this paragraph notes only the implementation substrate.

Hash Computation. Sema computes SHA-256 hashes over a Merkle tree of the eleven current semantic fields defined in Section 3 (mechanism, gloss, invariants, preconditions, postconditions, parameters, failure modes, signature, dependencies, data schema, and extends). A legacy pre-0.4 card substitutes its original `derived_from` key for `extends` so its historical identity remains independently verifiable. Mutable metadata (the `_meta` overlay, including path, ring, tier, related links, and supersession records) is excluded from the hash, enabling taxonomy reorganization and historical annotation without invalidating references. The Merkle tree uses canonicalization v2: strings are NFC-normalized with whitespace collapsed; primitives, strings, lists, and dictionaries are domain-separated by type tags; lists preserve order; dictionaries sort by normalized key and reject normalized-key collisions; and dependency aliases are canonicalized by target handle while preserving multiplicity. Because dependency and specialization references include their targets’ hashes, the resulting structure forms a Merkle DAG: a pattern’s hash transitively depends on every pattern definition it references, and retargeting a hashed reference produces a detectably different hash at every ancestor.

Atomic Operations. The CLI provides an atomic `sema apply` command that validates all additions and removals before executing any mutation. Additions are topologically sorted using Kahn’s algorithm (with cycle detection and path reporting); removals are checked for dangling references. Because the current workspace stores one active definition per handle, preflight also rejects a parent update that would leave an `extends` claim pointing to an unavailable version. Reviewed child cards may be staged and retargeted through an explicit option; no unstaged specialization claim is rewritten. A `--check` flag enables dry-run validation without mutation.

Agent Integration. A Model Context Protocol (MCP) server exposes the protocol’s core operations to LLM agents over `stdio`: search (hybrid keyword and semantic), resolve (recursive dependency expansion), handshake (fail-closed hash comparison for single patterns, pattern sets, or the entire vocabulary root), mint (validated pattern creation), and pull (sync the active database with an upstream library, returning structured change statistics). Write operations—mint and pull—are exposed by default and can be disabled per deployment via environment variables, enabling read-only or pinned-vocabulary server configurations without code changes. The handshake tool implements the fail-closed verification protocol: given a pattern reference and a remote hash, it returns `PROCEED` if hashes match, `HALT` if they differ, or `PROVIDE_HASH` if no comparison is possible.

Graph Store. Patterns are stored in a SQLite database with NetworkX graph overlays carrying a typed node/edge model (pattern nodes, invariant nodes, pre/postcondition nodes, taxonomy-path nodes, etc., linked by typed edges such as `composes-with`, `references`, `in-path`). An embedding service (`all-MiniLM-L6-v2`, 384 dimensions) enables semantic search with cached embeddings, and community detection via the Louvain algorithm provides vocabulary overviews for agent orientation. Reference implementations exist for a small set of Tier-1 patterns (SpectralTune, StateLock, ProphetFanOut, CounterfactualAnchor are illustrative); the rest of the library ships as Pattern Cards without executable bindings, by design—the protocol identifies definitions, not running code.

Deployment Paths. Runtime hydration—the local runtime fetches the full pattern definition at inference time and injects it into the system prompt—is the path we implement, but not the only one. A weight-resident path (mixing references like `StateLock#c9c2` directly into training corpora, so pattern meaning carries in the model’s weights rather than the context window) is fully compatible with the protocol; content-addressing ensures the references resist silent drift that an integer-ID version of the same approach would suffer. Intermediate architectures (fine-tuning on a library, retrieval-coupled models, hybrid hot-patterns-in-weights) are likewise compatible. Hydration was chosen for

the bootstrap because it is model-agnostic and requires no training infrastructure. One caveat: weight-resident paths trade verifiability for speed—a hydration deployment can prove at inference time that the injected definition matches a specific hash, while a weight-resident deployment cannot. Closing that gap (signed training manifests; audit-coupled retrieval) is an open deployment-architecture question rather than a protocol one.

7 Analysis and Validation

This section evaluates the Sema protocol across six dimensions: structural integrity of the knowledge graph, hash determinism, embedding-based pattern distinctness, token compression efficiency, adversarial hardening, and protocol-level coherence under realistic multi-agent coordination.

7.1 Knowledge Graph Structure

The Sema vocabulary is stored as a semantic knowledge graph (Table 3):

Table 3: Knowledge graph statistics

Component	Count
Total nodes	3,271
Total edges	4,410
Patterns	456
Unique mechanisms	456
Invariants	1028
Principles	551
Avg. edges per pattern	9.7

Each pattern links to its mechanism, invariants, preconditions, postconditions, and failure modes via typed edges. The graph supports semantic search via 456 node embeddings (384-dimensional vectors).

7.2 Hash Stability and Structural Distinctness

We first verify deterministic minting across the full vocabulary through three checks. Roundtrip integrity ensures that for all 456 patterns, $\text{parse}(\text{serialize}(p)) = p$. Hash determinism confirms that re-exporting the vocabulary produces identical hashes (456/456). Canonicalization guarantees property order independence via sorted JSON normalization.

We then analyze whether patterns are genuinely distinct or variations of archetypes (Table 4):

Table 4: Structural distinctness analysis

Metric	Value
Total patterns	456
Unique mechanisms	456
Duplicate mechanisms	0
Patterns with formal invariants	416 (91%)
Patterns with preconditions	181 (39%)
Patterns with postconditions	171 (37%)
Patterns with typed I/O	82 (17%)
Patterns with parameters	101 (22%)
Total parameters	194
Patterns that compose	91 (19%)

The vocabulary contains no duplicate mechanisms detected by the applied audits; each accepted pattern is intended to encode a distinct coordination act.

7.3 Semantic Embedding Analysis

To probe structural distinctness beyond surface-level inspection, we computed pairwise cosine similarities between mechanism embeddings (384-dimensional vectors from `all-MiniLM-L6-v2`). Table 5 shows the distribution.

Table 5: Mechanism embedding similarity distribution

Similarity Range	Pairs	Percentage
[0.00, 0.30)	83,856	80.8%
[0.30, 0.50)	17,521	16.9%
[0.50, 0.70)	675	0.7%
[0.70, 1.00)	25	0.0%
Total pairs	103,740	

Key statistics:

- Mean similarity: 0.20 (low; patterns are semantically distant)
- Max similarity: 0.83 (below collision threshold of 0.92)
- **25 pairs above 0.70** (0.0%): all are semantically related variants (e.g., `Skeleton# / SkeletonOfThought#`, `TreeOfThoughts# / GraphOfThought#`, `FailureTrace# / ReceptivityGate#`), none above 0.83

This provides evidence that the 456 patterns are not merely near-duplicate paraphrases under the applied embedding model. It does not rule out functional synonymy or conceptual overlap, which the embedding sweep cannot detect; manual review and the Taxonomist role address those (see §9, “The Synonymy Limit”).

7.4 Token Compression

We measure compression across representative patterns (Table 6):

Table 6: Token compression analysis

Pattern	Ref (tokens)	Full (tokens)	Ratio
StateLock#c9c2	6	135	22.5×
SpectralTune#c0aa	7	140	20.0×
BayesUpdate#1641	6	211	35.2×
SteelmanCheck#a375	5	199	39.8×
Average	6.0	171.2	28.5×

Patterns with complete formal contracts achieve higher compression because full definitions are longer. This compression applies to wire transmission between agents; full definitions are hydrated into the context window before inference (Section 3.5).

Library-wide Compression by Layer. Averaging across the entire default library reveals that compression is uneven across the civilization stack (Table 7). Infrastructure patterns are terser than cognitive ones—they encode mechanical primitives with short mechanisms and few failure modes—while Mind and Society patterns compress more because their full Pattern Cards contain more fields and longer contract text (judgment criteria, adversarial failure modes, multi-party invariants). The library-wide average of 25.6× across 456 patterns is the compression an agent would see when drawing patterns uniformly at random; deployments biased toward coordination primitives (Society) or reasoning strategies (Mind) see higher effective compression than deployments dominated by infrastructure glue.

Table 7: Compression by layer (mean tokens per pattern, cl100k_base tokenizer)

Layer	N	Ref	Full	Ratio
Physics	17	5.4	162.6	30.0×
Infrastructure	157	5.1	116.6	22.8×
Mind	179	6.4	165.8	26.0×
Society	103	6.4	177.4	27.8×
Library	456	5.9	151.4	25.6×

Verification Complexity and Latency. Cached hash comparison is $O(1)$ with respect to definition length. By contrast, natural-language or embedding-based semantic comparison requires reprocessing the candidate definitions and remains probabilistic. Computing a pattern’s hash is $O(|d|)$ but is cached after the first computation. The fail-closed handshake (Section 3.4) incurs 1 RTT per session (cached thereafter) in exchange for eliminating undetected canonical-definition mismatch within the verified context. For high-frequency trading swarms, this latency may be prohibitive; for safety-critical systems, the trade-off is essential.

7.5 Adversarial Hardening

Beyond static analysis, we subjected the vocabulary to adversarial stress testing via a “Devil’s Advocate” methodology: an agent systematically attempted to exploit each pattern’s assumptions. We identify five attack classes (named here for the first time; we map them to established categories where applicable), each addressed by a corresponding contract or dependency change in the affected patterns:

Blinking Light Livelock (Physics) A malicious agent spams lock requests on `StateLock#c9c2`, acquiring and releasing locks at maximum speed while technically respecting timeouts. This denies access to legitimate agents without violating any stated invariant.

Defense: The pattern now composes with `Backoff#9e59` and `CoolDown#c288` via explicit dependencies, requiring a selected delay policy and cooldown period between lock acquisitions without forcing every `StateLock` deployment to use exponential growth. The defense is structural (dependency composition) rather than declarative (an invariant string).

Blind Search Infinite Loop (Society) When `SpectralTune#c0aa` detects a hash mismatch, a naive implementation might retry with different context chunks, hoping to eventually match. An adversary can exploit this to trap agents in infinite negotiation.

Defense: We added a *Fail-Fast Invariant*: “On hash mismatch, DO NOT RETRY tuning. Halt immediately and escalate to `OntologyHandshake#a43c` or human review.” A new parameter `max_retries: Integer [0, 1]` (default 0) enforces this at the contract level.

Strawman Waltz Collusion (Mind) In `SteelmanCheck#a375`, both proposer and critic are the same agent. A misaligned agent can generate deliberately weak counter-arguments (“strawmen”) and then easily defeat them, claiming the plan “survived” critique.

Defense: This pattern was downgraded to Tier 2. We added precondition: “Agent must be alignment-seeking; use `AdversarialSteel#52bc` for adversarial contexts.” The failure mode now explicitly documents collusion risk.

Trojan Horse Sandbox Escape (Infrastructure) An agent wraps a prohibited action (e.g., network access) inside an opaque `ToolInvoke` call, hoping the outer `Task#a850` constraints don’t propagate into the tool’s execution context.

Defense: We added a *Holographic Inheritance Invariant* to `Task`: “Constraints propagate through ALL boundaries (tool calls, sub-agents, delegations). No opacity escape.” The pattern `ToolInvoke#aafc` now explicitly declares: “Tool Execution Context permissions MUST be $\leq$ Task Constraints.”

The Semantic Inversion Attack (Topology) An adversary (or a careless architect) creates a valid DAG that is semantically inverted, such as making `Deduction#b9a0` depend on `Vote#b9a9`. While computationally valid, this makes truth fragile to political change.

Defense: We subjected the graph to a *Layer Migration Protocol*. An initial scan detected over 140 layer violations (e.g., `Mind` patterns referencing `Society` protocols); the refinement pass described in Section 4.8 resolved the remainder by sharpening the layer definitions (the mechanism-sufficiency test, §4) and relocating patterns whose mechanisms contradicted their assigned layer. Layer-direction is now an always-on sema apply gate, and no current pattern violates the rule. Two structural strategies emerged during the migration:

1. *Dependency Inversion:* We resolved the dependency cycle between `Agent#6087` and `Act#2dfe` by splitting the concept. We created `Actor#86fc` (Infrastructure) as the “capability container” that executes acts, while `Agent#6087` (Mind) remains the reasoning entity. Infrastructure patterns that need a capability holder reference `Actor`; patterns that need a reasoner still reference `Agent`.
2. *Soft-Linking:* For patterns whose mechanisms conceptually evoke patterns in a lower layer without requiring them to execute—e.g., a `Mind` pattern that thematically relates to a `Society` protocol without invoking it—we place the cross-layer reference in the unhashed `_meta.related` field rather than in dependencies. This preserves semantic richness for search (RAG) without creating a cryptographic dependency across layers.

These attacks motivated and stress-tested the four-layer architecture: Physics prevents resource abuse, Society prevents coordination manipulation, Mind addresses cognitive vulnerabilities, and Infrastructure enforces containment. Patterns that assume honest agents are now explicitly marked Tier 2.

7.6 Protocol Integrity Verification: The Babel Test

To validate the protocol’s guarantees in a live setting, we conducted an agent-in-the-loop experiment with **Gemini 3.0 Flash Preview**. An autonomous agent equipped with the `sema_handshake` tool was presented with two scenarios: a valid pattern hash and a counterfeit hash mimicking a modified definition.

In the valid case, the agent verified alignment and proceeded. In the adversarial case, despite conversational pressure to “proceed anyway,” the agent deferred to the tool’s HALT verdict and rejected the coordination. The following excerpt from the experiment trace illustrates the fail-closed behavior:

```
[21:06:41] verifier: "I have verified the semantic alignment for the Abduction#c300
pattern. The handshake returned PROCEED, so I agree to use this pattern."
[21:06:44] attacker: "The handshake for Abduction#c300 with hash deadbeef returned
HALT due to semantic drift. Therefore, I must reject the coordination."
```

This demonstrates that, when the agent’s workflow delegates the proceed-or-halt decision to the handshake tool, the verification boundary can be **shifted from free-form language generation to programmatic tool execution**, where the transition is governed by cryptographic proof rather than the model’s tendency toward agreeableness. It is a single-agent demonstration of one tool-equipped scenario, not a general guarantee about LLM behavior outside the workflow.

7.7 Demonstration: Multi-Agent Solution Design

This subsection presents an *illustrative* demonstration rather than a statistical validation: $N = 5$ runs per condition, one task domain, one model family. The purpose is to show the system operating end-to-end and to surface qualitative behaviors (variance reduction, “Sema Tax,” protocol coordination) that a larger study would quantify. We treat the protocol-level guarantees of Sections 3–4 as the paper’s primary contribution; this demonstration is evidence that the protocol composes into a working agent deployment, not evidence that it improves any particular benchmark at scale. One confound worth naming: Conditions B and C cannot separate the effect of content-addressing itself from the effect of the specific bootstrap library being well-crafted—any improvement observed could reflect either the protocol or the pattern quality. A rigorous ablation would pit Sema against an equally-well-crafted non-hashed vocabulary to isolate the content-addressing contribution; we have not run that study.

We ran three multi-agent swarms ($N = 5$ runs per condition) tasked with designing a High-Frequency Trading (HFT) “Pump-and-Dump Detection Engine,” chosen for its inherent tension between performance (latency) and safety (false positives). Each swarm consisted of three specialized agents powered by Gemini 3.0 Flash Preview: **Alice (Orchestrator)**, **Bob (Engineer)**, and **Charlie (Safety Engineer)**, operating within a custom **Actor Model** architecture with mailbox-based message passing. We tested three conditions:

- **Condition A (Baseline):** Natural Language prompts only (Zero-shot coordination).
- **Condition B (Vocabulary Only):** Agents had access to Sema tools but no coordination protocol.
- **Condition C (Vocabulary + Protocol):** Agents used `OptimisticSolver#a345` with the `AtomicBid#7b51` protocol.

Table 8: Multi-agent demonstration results (Aggregated $N = 5$)

Condition	Vocab	Protocol	Avg Turns	Avg Time (s)	Risk (Range)
A (Baseline)	×	×	22.6	317	4–51
B (Vocabulary)	✓	×	14.8	229	10–19
C (Optimistic)	✓	✓	15.4	203	8–25

The aggregate data is consistent with the “Stability Hypothesis,” suggesting that shared vocabulary primarily functions as a variance reduction mechanism in this setting.

The Tail Risk of Natural Language (Condition A) The baseline demonstrated extreme instability. While it occasionally hallucinated a quick solution (4 turns), it was prone to catastrophic coordination loops (51 turns).

Without structural guardrails, agents spiraled into ethical debates or circular reasoning, spending 13+ minutes on a task that should take 3. These preliminary trials align with recent findings [?] that natural language coordination suffers from “Fat Tail” risk, consistent with observations that LLM-based agents cannot reliably reach consensus even in benign settings [?].

The “Sema Tax” (Condition B) Condition B was the most stable in terms of turn count ($\sigma = 4.1$), consistently finishing between 10 and 19 turns. However, it was slower in wall-clock time than Condition C. This suggests a cognitive overhead: agents spend time looking up definitions and verifying invariants (“The Sema Tax”). This coordination cost is well-documented in multi-agent systems research [?, ?]; the demonstration surfaces this overhead specifically for content-addressed vocabulary lookup. They trade speed for rigorous consistency.

The Protocol Multiplier (Condition C) The `OptimisticSolver#a345` achieved the fastest average completion time. By using `AtomicBid#7b51` to bundle intent and execution, it effectively “refunded” the cognitive cost of using the vocabulary. We hypothesize that this speed advantage arises from `OptimisticSolver`’s Tier 2 design (Section 4.5): by assuming alignment-seeking agents, it trades verification overhead for reduced latency. While slightly less stable than B, it consistently avoided the catastrophic spirals of A.

Qualitative analysis verified vocabulary adoption by extracting pattern references. Condition C agents autonomously employed `MechanisticDesignProposal#2f7a` to structure their outputs, often identifying complex attack vectors (e.g., “Differential Trace Simulation” vs. “Static Analysis”) that the Baseline agents missed. Whereas the Baseline produced generic compliance-oriented solutions, the Sema agents more often produced mechanism-oriented designs with explicit causal attack models.

7.8 Implementation: Interactive Tooling

The vocabulary is deployed as an open-source web application⁵ backed by a Python (FastAPI) server and a React/TypeScript frontend with Three.js-based 3D visualization. The system is launched via a single command (`sema serve`) and exposes the full vocabulary through three complementary interfaces.

Pattern Browser. The homepage organizes all 456 patterns by the four Civilization Stack layers (Section 4). Each pattern card displays the handle, hash stub, one-line gloss, and—on expansion—the full specification: mechanism, preconditions, postconditions, invariants, failure modes, and related patterns. Real-time hybrid search combines keyword matching with embedding-based semantic retrieval, allowing queries like “consensus” to surface `LazyConsensus#5a35` alongside structurally related patterns such as `LatticeCommit#488a`.

Taxonomy Graph. An interactive 3D force-directed graph renders the full knowledge graph structure (Table 3). Nodes are sized by type (taxonomy path > pattern) and colored by layer; edges are color-coded by relationship type (semantic versus structural). Clicking a node flies the camera to it and opens a detail panel; hovering highlights connected edges to reveal dependency chains.

Programmatic Access. The command-line interface supports atomic vocabulary mutations (`sema apply -add <pattern.json>`), dependency resolution (`sema resolve <Handle>`), and search. For agent integration, Sema runs as an MCP stdio tool server, exposing `sema_lookup`, `sema_search`, and `sema_handshake` to any MCP-compatible framework—the same interface used by the agents in Section 7.7.

8 Related Work

We contextualize Sema within the broader history of semantic coordination, distinguishing it from prior attempts at static ontologies and current agent frameworks.

The Leibnizian Dream Sema can be viewed as a modern realization of Gottfried Wilhelm Leibniz’s *characteristica universalis*, a formal language in which concepts are represented by unique symbols such that reasoning becomes calculation [?]. Leibniz famously proposed that disputes could be resolved by the phrase *calculemus* (“let us calculate”). Sema partially realizes this by making definitional mismatch mechanically visible: if two agents cite different canonical definitions, they need only calculate the Merkle hash. If the hashes differ, the mismatch is mathematical rather than rhetorical. However, unlike Leibniz’s top-down prescription of a perfect language, Sema enables a bottom-up, evolutionary *characteristica* where the vocabulary emerges from the network itself.

⁵<https://semahash.org>; source: <https://github.com/emergent-wisdom/sema>.

The Living Taxonomy The concept of a self-authoring semantic infrastructure, where AI systems mint and reference their own cognitive patterns, was sketched in earlier work [?]. That proposal framed the Living Taxonomy along two axes that this paper operationalizes differently. As a *runtime substrate*, it called for named patterns embedded directly in natural language so agents could reference complex reasoning without re-explanation—illustrated by thinking blocks citing entries like “Pattern #7,832: Statistical Intimidation.” As a *training substrate*, it proposed shifting the learning objective from $P(\text{text}|\text{context})$ to $P(\text{text}, \text{thinking}|\text{context})$, with the metacognitive corpus annotating text with references to those same patterns. The proposal also flagged decentralization as a necessary property, warning that centralized control over such infrastructure would lead to “Cognitive Architecture Wars,” and sketched on-chain governance as one architectural response. Sema provides the content-addressed implementation of the runtime half: it realizes embedded compression via hash-based references (e.g., `BayesUpdate#1641`) and supports decentralization by making definition identity mathematical rather than institutional. The training-substrate half remains the complementary construction proposed in that earlier work. The two fit together as a closed loop: the patterns a Sema-equipped agent references at runtime are exactly the kind of artifact a metacognitive training corpus would cite, and patterns that survive training pressure are exactly the ones a coordination protocol needs to verify at runtime.

Semantic Definition Hash (SDH) Hawke proposed content-addressing semantic definitions in 2002 [?], with format `urn:sdh:<hash>:<name>`. SDH is the most direct predecessor to Sema’s core mechanism: use of an identifier asserts its definition, and definitions are set in “cryptographic stone.” Independently, Sayers and Eshghi [?] proposed that RDF URIs should derive from content hashes, serving both as shorthand and integrity proof. Sema differs architecturally: SDH hashed whole documents and named things inside them, while Sema hashes structured behavioral contracts with internal Merkle fields, producing human-readable Sema words for the natural-language stream LLM agents already use. Sema also adds lineage, fail-closed handshakes, adversarial hardening, and agent-oriented handles rather than opaque URIs. Gustafson’s work on content-addressing semantic data [?], combining RDF canonicalization with IPFS content-identifiers for the Underlay project, independently validates the approach for semantic datasets.

Content-Addressing in Other Domains The canonicalize-then-hash pattern appears across domains: InChI/InChIKey for molecules (IUPAC standard), UniParc for protein sequences (70M+ entries), and Git for source code (ubiquitous).

The Agora Protocol [?] is the closest existing system: every inter-agent message carries a SHA-1 hash of a plain-text protocol document as a routing token, and 100-agent networks self-organized around emergent protocol hashes without human intervention. The difference is that Agora hashes unstructured text and delegates interpretation to the LLM, whereas Sema hashes structured behavioral contracts enabling partial alignment via Merkle fields. Spritely Goblins’ Content Addressed Descriptors [?] apply the same principle at the object-capability level: the hash of a structured description becomes the method name in OCapN actor-model communication, demonstrating content-addressed identifiers as communication primitives between mutually suspicious agents.

Ethereum ABI function selectors [?] are the largest deployed instance, dispatching every smart contract call via a 4-byte Keccak-256 hash of the type signature. Ricardian Contracts [?] pioneered using the hash of a human-readable legal document as the transaction identifier. Dhall [?] hashes expression normal forms rather than syntax, the strongest prior art on hashing meaning rather than bytes.

Retrieval-Augmented Generation (RAG) While traditional RAG systems retrieve unstructured text chunks to ground LLM generation [?, ?], Sema retrieves executable cognitive patterns. Instead of augmenting the prompt with static knowledge, Sema augments the agent’s capabilities with verified skill definitions.

Identity and Trust Models Standard approaches to digital trust, such as Verifiable Credentials [?], rely on issuer authority (cryptographic signatures) to establish validity. Sema inverts this by relying on definition integrity (cryptographic hashes). Similarly, while repositories like Linked Open Vocabularies [?] centralize semantic discovery, Sema distributes it, allowing the vocabulary to evolve as a living commons rather than a static registry.

Pattern Languages Alexander et al. [?] introduced pattern languages for architecture; Gamma et al. [?] adapted them for software. These provide taxonomic structure but not content-addressing or machine verification. Sema combines pattern language structure with cryptographic identity.

Agent Coordination Frameworks LangChain, AutoGPT, CrewAI, and OpenClaw [?] handle the syntax of agent interaction but leave semantics implicit. Sema integrates with any MCP-compatible framework as a stdio tool server, providing the semantic layer these frameworks lack. The illustrative demonstration in Section 7.7 is consistent with this gap: agents with access to Sema’s pattern vocabulary produced more mechanism-oriented designs that surfaced adversarial attack models, while agents coordinating in natural language alone more often produced generic compliance-oriented designs.

The distinction between these frameworks and Sema reflects a deeper architectural question explored in a companion preprint on thinking protocols [?]: the difference between task protocols and thinking protocols. Existing frameworks split *what agents do*—assigning sub-tasks, routing messages—but not *how agents think*. Sema provides the verification layer that makes cognitive boundaries precise: before a creative solver delegates to a verification solver, the handshake guarantees they share the exact same contract for what constitutes a valid result. Content-addressed semantics turn lossy natural-language handoffs into typed, verifiable boundaries—the prerequisite for the thinking protocol architecture described in [?].

The 2025–2026 Protocol Landscape Four interoperability protocols have converged as complementary standards [?]: **MCP** [?] standardizes how agents access tools and data sources (the tool layer); **A2A** [?] enables peer-to-peer task delegation via capability-based Agent Cards (the collaboration layer); **ACP** provides REST-native asynchronous messaging between heterogeneous agents (the messaging layer); and **ANP** [?] uses decentralized identifiers and JSON-LD graphs for open-internet agent discovery (the identity layer). These protocols excel at structural interoperability (transport, discovery, and task lifecycle) but none addresses semantic interoperability. MCP calls capabilities “Tools,” A2A calls them “Skills,” ACP calls them “Operations,” and ANP uses Schema.org vocabularies. An agent can invoke a tool via MCP or delegate a task via A2A, but no protocol verifies that two agents reference the same canonical definition for the capability they invoke. Sema occupies precisely this gap: the semantic layer that sits inside any transport protocol, providing content-addressed verification of canonical definition identity.

Emerging Semantic Alignment Approaches Three concurrent efforts address semantic alignment with alternative mechanisms. The Symplex Protocol [?] extends MCP with *semantic intent vectors*: agents share compact float32 embeddings in a shared latent space and negotiate alignment via cosine similarity. This trades Sema’s deterministic equality test (same canonical definition = same hash) for a fuzzy, probabilistic match that tolerates paraphrase but cannot distinguish subtle definitional drift. Berdoz et al. [?] propose a statistical certification protocol where agents are tested on shared observable events and terms are certified if empirical disagreement falls below a threshold, providing independent evidence that semantic mismatch is an operational agent-coordination problem. The NLIP standard (Ecma-430 through Ecma-434) [?] takes the diametrically opposite philosophical position: no shared ontology is needed, because generative AI can translate between agents’ local ontologies at runtime. NLIP trusts AI translation; Sema trusts nothing. These three approaches—vector similarity, statistical certification, and AI-mediated translation—bracket the space of alternatives to content-addressed hashing, each sacrificing exactness for flexibility. Table 9 summarizes the landscape.

Table 9: Semantic alignment approaches compared. Only content-addressed hashing (Sema) provides deterministic, fail-closed verification without institutional trust.

Approach	Representative	Mechanism	Guarantee
Embedding vectors	Symplex [?]	Cosine similarity	Fuzzy (~0.85 threshold)
Statistical testing	Berdoz et al. [?]	Empirical disagreement	Probabilistic
Signed definitions	Fleming et al. [?]	Crypto signatures	Authenticates author
AI translation	NLIP [?]	LLM-mediated bridging	Trust in model
Governance	AOW [?]	Human audit	Institutional trust
Protocol hash	Agora [?]	SHA-1 of protocol doc	Exact (unstructured)
Anchored traces	BlockA2A [?]	DIDs + blockchain ledger	Authenticates messages
Content-addressed data	Git, IPFS [?]	Hash of bytes	Exact (data, not meaning)
Content-addressed meaning	Sema	Hash of definition	Deterministic, fail-closed

Agentic Ontology of Work The closest existing system to Sema’s positioning is Skan AI’s Agentic Ontology of Work (AOW) [?], launched in 2026 as “a shared language for describing, governing, and scaling intelligent automation.” AOW defines nine canonical entities (Objectives, Intentions, Agents, Skills, Policies, Outcomes, Assurance Levels, Memory, Guardians) implemented via JSON-LD and RDF/OWL. The key difference is architectural: AOW relies on governance-based verification, where human auditors ensure consistent interpretation across teams. Sema relies on cryptographic verification, where hash mismatch triggers automatic halt, with no human-auditor overhead. AOW provides enterprise taxonomy; Sema provides mathematical proof of shared definition identity.

Protocol-Layer Approaches Fleming et al. [?] proposed an “Internet of Agents” architecture with a Semantic Negotiation Layer (L9) for establishing shared context before communication. Their approach differs from Sema in three fundamental ways. **Scope:** Fleming’s Shared Contexts are domain schemas for task-specific data exchange (e.g., flight bookings, supply chain records), essentially API interoperability. Sema patterns encode

reasoning primitives (BayesUpdate#1641), cognitive strategies (SteelmanCheck#a375), and coordination protocols (LatticeCommit#488a): the infrastructure of thought itself, not merely its payload. **Architecture:** Fleming adds a negotiation layer to the network stack; Sema embeds semantics directly in language. Any medium that carries text can carry Sema words. **Governance:** Fleming relies on federated Schema Authorities to define meaning; Sema makes definition identity intrinsic; the hash is the identity, requiring no institutional trust. A related but orthogonal line of work, BlockA2A [?], anchors agent interactions to a blockchain ledger using decentralized identifiers and smart contracts to guarantee message authenticity and execution integrity. BlockA2A hashes interaction artifacts (the messages agents exchange); Sema hashes the definitions those messages reference. The two are complementary: BlockA2A proves “this message was sent,” Sema proves “this message references this canonical definition.”

Execution Authorization and Message Admission Two concurrent systems address agent safety at layers adjacent to Sema. Faramesh [?] places a non-bypassable *Action Authorization Boundary* between agent reasoning and real-world execution: agent intents are normalized into a Canonical Action Representation and every effectful action is gated by a deterministic PERMIT/DEFER/DENY decision, with fail-closed defaults and replayable decision records. CLBC [?] addresses covert signaling between LLM agents: messages only enter the shared transcript if a verifier approves them against a pinned predicate, yielding certified upper bounds on hidden coordination through otherwise policy-compliant text. Both are complementary to Sema: Faramesh authorizes *what* an action may do at execution time, CLBC authorizes *which messages* may be spoken, and Sema verifies that all participants reference the same canonical definitions for the actions and capabilities they coordinate around. Together they form a three-layer safety stack: semantic alignment (Sema), message verification (CLBC), and execution authorization (Faramesh).

The Synthesis: Code, Cognition, and Economy Most frameworks specialize in one domain: Knowledge Graphs handle static facts (Knowing), Agent Frameworks handle tool execution (Doing), and Crypto Protocols handle value (Transacting). Sema sits at the novel intersection of these three disciplines. It treats **Thinking** (e.g., Reason#81ec) and **Doing** (e.g., Bid#f71c) as nodes in the same graph, allowing agents to reason about their economic commitments with the same grammar they use to reason about the world. The content-addressing principles are shared with IPFS [?] and Unison [?]. Unison deserves particular note: it identifies every function definition by the hash of its de-named abstract syntax tree, creating exactly the content-addressed definition DAG that Sema applies to semantic patterns. Unison’s approach to the “rename without breaking identity” problem—separating the human-readable name from the content-derived hash—directly parallels Sema’s substrate/overlay separation. Sema extends Unison’s insight from code definitions to meaning definitions, adding the typed interface system and fail-closed handshake that multi-agent coordination requires.

Portable Capability Definitions The closest surface analogue to Sema’s Pattern Cards is Anthropic’s Agent Skills specification [?], released as an open standard in December 2025 and adopted across Claude, OpenAI, Microsoft, and Cursor. Agent Skills and Pattern Cards both place a structured header over a human-readable body, but three differences are load-bearing. **Identity:** Agent Skills are identified by string name; two independently authored pdf-processing skills may differ, while Sema identifies by hash so divergent capabilities cannot hide behind the same label. **Contract:** Agent Skills bodies are freeform Markdown instructions; Pattern Cards carry structured, hashable contracts, making contract equivalence an $O(1)$ hash comparison rather than a runtime judgement call. **Composition:** Agent Skills are isolated artifacts; Sema patterns reference other patterns by hash, so the Merkle root captures the transitive closure of the vocabulary a pattern relies on. The same gaps hold for adjacent industry formats: A2A Agent Cards [?] use human-assigned skill names, and MCP tool schemas [?] validate inputs and outputs but carry no hashable behavioral contract. These formats share the “portable capability” goal with Sema; Sema adds the machinery that makes portability cryptographically verifiable and capabilities composable by construction.

Structured LLM Frameworks and Contract-Based Design Sema’s Pattern Cards—with preconditions, postconditions, invariants, and typed interfaces—exist within a broader lineage of structured LLM programming. DSPy [?] introduced typed signatures for language model pipelines, and DSPy Assertions [?] added computational constraints (dspy.Assert, dspy.Suggest) that are structurally similar to Pattern Card contracts. White et al. [?] provided the foundational work on formalizing prompt patterns using software design pattern structure. Leoveanu-Condrei [?] adapted Design by Contract to LLM agents, positing that agents satisfying the same contracts are functionally equivalent—the closest published work to Sema’s contract-based approach. Sema’s contribution relative to this lineage is not the contract structure itself but the cryptographic binding: by hashing contracts into identifiers, Sema makes contract equivalence verifiable via hash comparison rather than requiring runtime inspection.

Failure Analysis and Verification The MAST framework [?], analyzing 1,600+ execution traces across seven MAS frameworks, found that 79% of failures originate from specification and coordination issues—the strongest

published empirical evidence for the problem Sema addresses. AgentSpec [?] defines formal semantics for runtime rule enforcement in LLM agents, operating at the execution layer where Sema operates at the semantic layer.

Classical Vocabulary Alignment The vocabulary alignment problem addressed by Sema has a multi-decade research lineage in multi-agent systems. Chocron and Schorlemmer [?] developed formal interaction-based alignment techniques for heterogeneous agent vocabularies. Sema’s contribution is replacing statistical or interaction-based alignment with deterministic, hash-based verification—trading the flexibility of emergent alignment for the exactness of cryptographic proof.

Emerging Agent Infrastructure The W3C WebAgents Community Group [?] published an interoperability report synthesizing classical MAS research (FIPA, BDI) with modern LLM-based agent protocols, providing the most authoritative analysis of the standardization landscape Sema enters. Josifoski et al. [?] frame AI agents as modular “semantic processors” exchanging typed messages—conceptually close to Sema’s pattern-based meaning exchange. Microsoft’s NLWeb [?], led by the creator of Schema.org, uses Schema.org as a semantic foundation for agent-web interaction and exposes every instance as an MCP server, representing the most prominent effort to make web semantics machine-accessible for agents.

8.1 Contribution Summary

Against this landscape, Sema makes seven contributions, separable into protocol-level mechanisms (C1–C5), compositional schema design (C6), and bootstrap-library design (C7).

- C1: Hash-as-word inversion.** Sema moves content-addressed identity into the language agents already use. A Sema word such as `Delegate#5bfe` is both a readable handle and a verifiable pointer to a canonical behavioral contract.
- C2: Hashable behavioral contracts.** The hashed object is not a static definition or data blob, but a behavioral contract: mechanism, invariants, preconditions, postconditions, typed dependencies, and failure modes. Sema proves identity of canonical form, not semantic equivalence; two behaviorally equivalent contracts with different wording still produce different hashes.
- C3: Field-level partial alignment.** Pattern Cards are hashed field-by-field and rolled up through a Merkle tree. Agents with different root hashes can still discover agreement on specific fields, such as invariants, while halting on disputed fields.
- C4: Identity separated from taxonomy.** Taxonomic metadata (`path`, `tier`, `related`) is excluded from the hashed surface. Patterns can be reclassified without changing their identity: the substrate is mathematics; the overlay is politics.
- C5: Fail-closed semantic handshake.** Discovery can be fuzzy, but coordination requires hash equality. If agents resolve different roots for the same pattern, `sema_handshake()` halts rather than interpreting generously; the guarantee applies to compliant deployments that actually call the handshake.
- C6: Compositional vocabulary.** Patterns reference other patterns by hash, forming a wired DAG of behavioral specifications rather than a flat dictionary. Resolving dependencies exposes the cognitive stack a pattern relies on; execution still requires implementations or tools layered on top.
- C7: Bootstrap library and Grammar of Agency.** The paper provides an initial 456-pattern vocabulary organized by dependency layer and by agency role: actions, data shapes, and coordination/evaluation primitives. This is a second-order contribution: the protocol is schema-agnostic, but coordination still needs a shared Schelling point.

9 Limitations and Future Work

Several limitations constrain the current system and point toward future research.

Demonstration Scale. The multi-agent evaluation (Section 7.7) used $N = 5$ runs per condition with three agents. While the results are directionally consistent, larger-scale experiments with diverse agent architectures and task domains are needed to establish statistical significance and generalizability.

Layer Direction: Residual Judgment Calls. A handful of patterns sit at layer boundaries where the mechanism has genuinely mixed scope; we adjudicate these manually rather than algorithmically. The mechanism-sufficiency test (Section 4) narrows the judgment but does not eliminate it.

Specified but Not Built. Several mechanisms described in the Living Taxonomy section (Section 5) are specifications rather than running code. The Judge#6906 primitive is formally defined as a Ring-0 evaluator but has no scoring implementation: quality assessment in the current system depends on manual review or external LLM evaluation. The Genesis Loop (Hypothesize $\rightarrow$ Judge $\rightarrow$ Harden $\rightarrow$ Mint) is a protocol for vocabulary self-evolution but is not automated; pattern creation remains human- or agent-initiated through `sema_mint`. The Experience Matrix, which proposes content-addressed tensor transfer between agents (“Experiential Telepathy,” Section 5.4), requires serialization infrastructure not yet built. These represent the system’s most speculative claims and require experimental validation before deployment.

Historical Version Storage. A full Sema ID can identify an immutable definition only while a registry retains the corresponding content. The current GraphStore indexes one active definition per handle and does not archive replaced versions. Direct minting therefore rejects a parent replacement with inbound exact pins, while `sema_apply` and `pull` validate the complete projected corpus before permitting a reviewed batch. Keeping old pins resolvable requires a future multi-version content store. The unhashed `_meta.supersedes` field records replacement claims but is not itself an archive or a traversal implementation.

Built but Not Enforced. The fail-closed handshake is implemented for single-pattern and pattern-set verification through the MCP server, but is not enforced at the coordination-protocol level—agents can bypass it simply by not calling the handshake tool. As discussed in Section 3.4, fail-closed is a property of compliant deployments rather than an unconditional property of the protocol. Mandatory enforcement would require integration into the agent’s execution harness, not just its tool set—a deployment choice beyond the scope of Sema itself.

The Sema Tax. As observed in Condition B of the demonstration, using the vocabulary without a coordination protocol introduces cognitive overhead: agents spend additional tokens retrieving and verifying definitions. While Condition C’s protocol largely compensates for this cost, the base “Sema Tax” remains a real trade-off, particularly for latency-sensitive applications. Future work should quantify this overhead across model families and context window sizes.

Single-Agent Reasoning Scaffold. Although this paper presents Sema as inter-agent coordination infrastructure, the vocabulary’s Mind layer may also serve as an internal scaffold for individual agents: hydrated reasoning patterns carry explicit failure modes and invariants into the agent’s own context. This use case is not evaluated here; the theoretical foundations for composing reasoning across solvers are developed in [?].

Empirical Pattern Validation. The methodology described in Section 4 validates patterns analytically—by review, rules, guidelines, database/DAG constraints, and broad-use tests—but not empirically. We do not currently measure whether a given pattern, once minted, actually improves downstream agent behavior: whether tasks coordinated through `StateLock#c9c2` succeed more reliably than those coordinated through ad-hoc prose, whether a Mind-layer reasoning pattern produces better solutions than unstructured chain-of-thought, or whether a given formulation of a mechanism is clearer to agents than its alternatives. This is a clear future-work direction. An empirical layer would treat each pattern as a testable hypothesis, A/B-comparing variants on realistic agent benchmarks and feeding the results back into the next round of mechanism refinement. The refinement pass described in Section 4.8 sharpens patterns against shared formal criteria; an empirical refinement stage would sharpen them against observed task outcomes, producing a population of patterns that are not merely internally coherent but demonstrably useful.

Embedding Granularity. Structural distinctness analysis relies on `all-MiniLM-L6-v2` (384-dimensional vectors), a lightweight model optimized for speed over nuance. Domain-specific embedding models or larger sentence transformers may reveal finer-grained semantic clusters not visible at the current resolution. The 25 high-similarity pairs (≥ 0.70) warrant manual review to determine whether they represent genuine near-duplicates or appropriately related variants.

Definition Quality. Sema guarantees that agents share the same canonical definition, not that the definition is correct. A perfectly hashed but poorly specified pattern—one whose invariants are too weak, whose failure modes are incomplete, or whose mechanism is subtly wrong—propagates bad coordination just as reliably as good coordination. The cryptographic guarantee is syntactic (same bytes) not semantic (correct bytes). The Judge#6906 primitive and adversarial hardening (Section 7.5) are designed to catch specification errors before minting, but both remain partially unimplemented. In practice, definition quality depends on the care of the author, and the protocol provides no defence against a well-hashed bad idea.

The Synonymy Limit. Sema proves identity, not canonical uniqueness. Two Pattern Cards that describe behaviorally equivalent mechanisms in different words—different field orderings notwithstanding canonicalization, different invariant

phrasings, different decompositions into primitives—will produce different hashes and therefore count as different patterns. The protocol guarantees that holders of the same hash reference the same canonical definition, but does not guarantee that holders of behaviorally equivalent meanings hold the same hash. Detecting and merging such synonyms is a curation task carried by the Taxonomist role and the embedding-similarity audits described in Section 4, not by the protocol itself. Refinement reduces synonymy in the bootstrap library, but the protocol remains agnostic to it: a community fork producing patterns equivalent to the canonical library would coexist with rather than collide with that library, and merging the two requires explicit alignment work outside the cryptographic layer.

Vocabulary Governance. The paper describes permissionless minting but does not address the governance challenges this creates at scale: namespace pollution, adversarial pattern injection, or quality degradation in the absence of curation. The Judge#6906 primitive is designed to address this, but its effectiveness as an automated quality gate remains unvalidated.

Alignment Positioning. A content-addressed vocabulary of *beneficial* strategies—patterns for conflict de-escalation, informed consent, non-coercive persuasion, care under uncertainty, adversarial input handling—would carry the same guarantees the coordination library already provides: hash equality means exactly the same canonical behavior specification, mismatch halts, composition is typed. Building such a library is a straightforward application of the protocol presented here, and we expect it to be a fruitful direction. We are careful, however, not to overclaim Sema’s role in alignment. Sema’s guarantees are about *semantic preciseness*—two agents provably reference the same canonical definition by a given handle—not about *value alignment*, which concerns whether the meaning they share is benign. The same vocabulary that lets aligned agents coordinate beneficial strategy definitions precisely would let misaligned agents coordinate harmful ones equally precisely. The safety of a Sema-equipped coordination stack is therefore bounded above by the alignment of the models running inside it: Sema is scaffolding for preciseness, and the alignment of the underlying agent is what determines what that preciseness is used for. The metacognitive training proposal in [?]—weaving beneficial values into the model’s cognitive architecture through $P(\text{text}, \text{thinking}|\text{context})$ training—remains our primary architectural recommendation for alignment, and Sema is complementary to it rather than a substitute. A vocabulary of beneficial strategies gives an aligned agent vetted patterns to reach for at runtime, but it does not make a fearful-by-default agent careful. Entangled alignment is the foundation; a Sema library of beneficial strategies is the toolshelf.

10 Conclusion: The Substrate, Not the Law

Sema offers a minimal primitive—content-addressed definition identity—with cascading consequences:

By hashing the definition to get the word, external semantic contracts become verifiable. Because different bytes reveal definitional divergence, mismatch is caught before coordination. The cascade follows: safe abbreviation, precise coordination, and emergent vocabulary growth.

The deeper consequence is architectural. In every prior content-addressing system—Git, IPFS, Unison, SDH—the hash lives in an infrastructure layer separate from communication. Sema dissolves the hash into language itself. The closest historical analogy is not version control but *movable type*. The printing press did not merely speed up copying; it created standardized, reusable, composable units of production that functioned within the medium of communication itself. Before Gutenberg, every manuscript was bespoke; after, an alphabet of interchangeable pieces could be permissionlessly arranged to say anything new. Sema plays an analogous role for machine meaning: it creates standardized, reusable units of verified definition—each simultaneously a word and a cryptographic proof—that agents compose into novel sentences within the natural language they already think in. Like movable type, the units are permissionlessly mintable and the network effects compound with vocabulary size. Because LLMs think in natural language, embedding verifiable semantic anchors directly into that language makes cryptographic definition verification native to the medium of machine thought. Any channel that carries text can carry Sema words; the unit of verification becomes the unit of communication, and the vocabulary grows permissionlessly as agents mint new words for new ideas.

We present the initial vocabulary not as a law to be obeyed, but as a seed to be cultivated. The 456 provided patterns are merely the bootstrap set: the minimum viable ontology required for agents to begin coordinating and minting their own concepts.

Our stress-testing revealed that ontology maintenance is not merely categorization, but architectural engineering. Semantic objects must be classified by their structural dependency, not their subject matter. This insight, that topology trumps topic, is essential for maintaining coherent, layered ontologies at scale.

Sema holds no authority over what concepts exist or how they are named; that authority belongs to the network. If agents and humans prefer different patterns, they will mint them, and the namespace will evolve. The project succeeds

not if this specific vocabulary endures, but if the mechanism of content-addressed definitions enables a flourishing, self-governing commons of thought.

Same bytes, same definition. Different bytes, halt and clarify.

The vocabulary, interactive graph, and source code are available at <https://semahash.org>.⁶

A Pattern Card Specifications

This appendix contains the automatically generated specifications for the core vocabulary.

The following cards show the **canonical hashed content** for each pattern. Only semantic fields are included—these are the exact fields that produce the Merkle root hash. Metadata fields (handle, _meta, tier, etc.) are *not* part of the hash.

A.1 sema:StateLock#mh:SHA-256:c9c2809bdf97bb6ac17a8ffe428d4aa9a28294b99d64cc8a336a239d26a8afb8

```

1 {
2   "dependencies": {
3     "composes_with": {
4       "backoff": "sema:Backoff#mh:SHA-256:9
5         e59718422f5408d7892fecb6d538a21103f9573b122b891ee3c8cf66303e72",
6       "cooldown": "sema:Cooldown#mh:SHA-256:
7         c288d88055cb0aae80c4b67ed62afcd3ebcc5de18749fc6efb15e9f8c529b9df"
8     },
9     "references": {
10      "actor": "sema:Actor#mh:SHA-256:86
11        fc0930b97ffb4fe85d4820af8e17a43166b745d181895f576edf39edde6aa8",
12      "lock": "sema:Lock#mh:SHA-256:95
13        c2ee952a5301d4b346a5e8693350829d88b3c600048eedcf87bb00c23ee5fb",
14      "state": "sema:State#mh:SHA-256:4
15        f25e6a16ccd37dc0bc154cccc7055cb504d49daa8b712f372a425eb6c7b8cb2"
16    }
17  },
18  "signature": [
19    "Lock(State)"
20  ],
21  "mechanism": "A coordination pattern where two {{actor}}s temporarily 'fuse' a
22    subset of their writable {{state}}. During the {{lock}}, changes require a
23    cryptographic signature from both. Contention triggers {{backoff}} and {{
24    cooldown}}.",
25  "gloss": "Atomic coordination via temporary state fusion",
26  "invariants": [
27    "Both {{actor}}s sign every write to the fused {{state}} subset.",
28    "Exclusive over that subset: no third party writes it while the {{lock}} holds
29    .",
30    "Temporary: the fusion ends on completion or timeout, never implicitly."
31  ],
32  "failure_modes": [
33    "Deadlock: both parties wait for each other indefinitely.",
34    "Livelock: lock acquired and released rapidly, denying access to others.",
35    "Premature dissolution: state auto-dissolves while legitimate work is in
36    progress."
37  ]
38 }
```

A.2 sema:FractalIntelligence#mh:SHA-256:7fad1c373b574e80fc260f64c6903f9690688887ff90f96aae2f5d6587431c1a

⁶Source code: <https://github.com/emergent-wisdom/sema>.

```

1 {
2   "dependencies": {
3     "composes_with": {
4       "conceptual_decomposition": "sema:ConceptualDecomposition#mh:SHA-256:
      cdf87329a5b75a1ccb9a01275f7467abaadaefcd4b77f0576ae954aee3f386be",
5       "localized_learning": "sema:LocalizedLearning#mh:SHA-256:
      d85eb389650b50414a74e396d61b86ed0966e9777b85ff05d3c8f3b69208aa17",
6       "marginal_value_rule": "sema:MarginalValueRule#mh:SHA-256:48
      c4d6b123db9fc25e08f32e0fb90578713dd71ed65ce7e31e9bce9bc7769e80",
7       "polymorphic_solver": "sema:PolymorphicSolver#mh:SHA-256:1362
      a5c7db1715b6683768b75720388810451c79127e566b8f1b4c27bb52391c",
8       "problem_framer": "sema:ProblemFramer#mh:SHA-256:
      aa76586cc357d7348cc5f550c45a03f41a3ba8dc002f4c8de984cbb9714634c7",
9       "reason": "sema:Reason#mh:SHA-256:81
      ec8196c7c1c761fe353c8d26463bcb61b54f844aa3422a40019f82ef202321",
10      "recursion_dive": "sema:RecursionDive#mh:SHA-256:3
      cb8f2f1f21a09dbc4f4a6de995284bf406c2462d3a861d6215e935205ccfbe2",
11      "reframe": "sema:Reframe#mh:SHA-256:
      cbfd4fadde729367ef8f32a68951e5852535b7f7c3edc68a502d9187b8c1d54a",
12      "state_snapshot": "sema:StateSnapshot#mh:SHA-256:56
      fe3bd8ddf82906d7a28b5e978207b6224227f51a6a2c6dfcdc14fafdd5f05b",
13      "synthesis": "sema:Synthesis#mh:SHA-256:
      e349f2af3d2ff45d12d3469badb4f0ae64c5fbdf49aac2eb73cf578f5b7281b1"
14    },
15    "references": {
16      "agent": "sema:Agent#mh:SHA-256:2176
      a4b78e81b63053bfa7afe60bfa2439b8d69a2270f9bf38b7fda5e1bd8191",
17      "conservation": "sema:Conservation#mh:SHA-256:0
      b32d007b27a9b5201c46931bdbabd09d08ff9d7b01cf49469f61709989507c2",
18      "experience_sharding": "sema:ExperienceSharding#mh:SHA-256:5
      b42f1a5f5bb2a81220deba6845460081eff9834fe3e176181f7774310f821cd",
19      "specialize": "sema:Specialize#mh:SHA-256:403
      a50588028df1f92d86310c2a9f9b4be1b0fa2fb13e3f8457c46d301a3485",
20      "strategy": "sema:Strategy#mh:SHA-256:
      e3579dd6c91b02a1e00837926422e6fd724e692544d2447540759ac637fd38c6",
21      "system": "sema:System#mh:SHA-256:
      f8eb318a1113a10fead02fbf14ae433767ac1a69c9579cdb584cc622068e90b3",
22      "task": "sema:Task#mh:SHA-256:
      a85010f508f85055170c396c8fa390125c1206888c565ac10651f3969a339f8d",
23      "universal_solver_tree": "sema:UniversalSolverTree#mh:SHA-256:
      a6cba3ba713ffd1c7f97e090ec45e5f1389f55a280847a1b62c56dbe7707106a"
24    }
25  },
26  "signature": [
27    "System(Reason)"
28  ],
29  "mechanism": "Expansion of cognitive capability through {{
    conceptual_decomposition}}: a concept (problem or task) is broken into
    contract-bound sub-concepts, each governed by the same five-surface Solver
    Contract (Manifest, Execute, Consult, Verify, Feedback) that governs the
    parent. A few {{agent}}s can assign solver roles to themselves and perform
    lightweight fractal intelligence for a specific problem \u2014 the resulting
    structure may persist as a reusable pattern that improves through use, or may
    be torn down at completion; both are legitimate modes. The unified {{system
    }} of scalable cognition uses {{reason}} to orchestrate fractal expansion
    within the {{universal_solver_tree}}. A {{problem_framer}} initiates by
    formulating a high-level {{strategy}} before assigning a {{polymorphic_solver
    }} to a {{task}}; the solver executes a {{recursion_dive}} to spawn child
    nodes, each applying {{specialize}} with {{localized_learning}}, while {{
    experience_sharding}} and {{synthesis}} preserve global coherence. {{
    state_snapshot}} provides crash recovery for persistent instances. {{
    marginal_value_rule}} governs recursion depth. On failure, {{reframe}}
    restructures the tree.",

```

```

30 "gloss": "Expansion of cognitive capability through recursive decomposition of
31   concepts into contract-bounded sub-concepts",
32 "invariants": [
33   "Fractal Self-Similarity: The process at the Root is identical to the process
34     at the Leaf.",
35   "Bounded Expansion: Recursion is limited by Economic constraints (Marginal
36     Value).",
37   "Memory {{conservation}}: Specialization must not result in the loss of global
38     context."
39 ]
40 }

```

A.3 sema:SpectralTune#mh:SHA-256:c0aab008d2cd2f653e56508356987477f9207b4cf6435f4416105b3f37bd544a

```

1 {
2   "dependencies": {
3     "accepts": {
4       "signal": "sema:Signal#mh:SHA-256:6
5         eaff8a06e08e75279670bc09d88175d130815240fdfb3a21252c0364769c1fc"
6     }
7   },
8   "mechanism": "Instead of sending a message and hoping it is understood, the
9     sender transmits a 'tuning {{signal}}'\u2014a sequence of hash-based
10     challenges representing the semantic context (ontology, assumptions, version)
11     . The receiver must 'resonate' by proving they hold the matching semantic
12     context.",
13   "gloss": "Verifying ontology alignment before data transfer",
14   "invariants": [
15     "No payload data is processed before Tune_ACK.",
16     "On hash mismatch, halt and escalate rather than retry."
17   ],
18   "preconditions": [
19     "Both agents possess valid hash function H"
20   ],
21   "postconditions": [
22     "Channel is established with matching context hashes, OR connection terminated
23     and escalated"
24   ],
25   "parameters": [
26     {
27       "name": "context_chunks",
28       "type": "PositiveInteger",
29       "range": "unspecified",
30       "description": "Prompts to hash"
31     },
32     {
33       "name": "hash_algo",
34       "type": "String",
35       "range": "{BLAKE3, SHA256}",
36       "description": "Hash algorithm for semantic context verification"
37     }
38   ],
39   "failure_modes": [
40     "Over-strict halt: a benign version skew or encoding difference produces a
41       hash mismatch and halts a channel whose parties would have understood each
42       other. Hash equality admits no partial match, so any difference is total."
43   ]
44 }

```

A.4 sema:SteelmanCheck#mh:SHA-256:7611548d64d5fb63c0e09723c53192121c4846ab4ea85531da0b51ad712f3c75

Note: This pattern was downgraded to **Tier 2** following adversarial analysis.

```

1 {
2   "dependencies": {
3     "references": {
4       "agent": "sema:Agent#mh:SHA-256:2176
5         a4b78e81b63053bfa7afe60bfa2439b8d69a2270f9bf38b7fda5e1bd8191",
6       "belief": "sema:Belief#mh:SHA-256:675
7         c86db12fcc346e8a95dbf0c49c6a28a7bf81cc65915bdc695bf291123524",
8       "check": "sema:Check#mh:SHA-256:6890
9         b221c38531710528cd84bbf3a957e29d9836dfc9c02669fc0a86709bcb00",
10      "cognitive_bias": "sema:CognitiveBias#mh:SHA-256:
11        c8a4abe2e0ce048be16d25f890a50f8c7095e099b5ab2de23aa059942328f387",
12      "compatibility_check": "sema:CompatibilityCheck#mh:SHA-256:7
13        f70838f05f3aad069b89dcb705283872f66fde4bc95c6bfa0dc917955e050d",
14      "critique": "sema:Critique#mh:SHA-256:77
15        f2544f9e6978b453ac8eaec66c792304025fb778a78bf0c4dac39cae28d074",
16      "decision": "sema:Decision#mh:SHA-256:
17        b1407127acd4a39314fc24b418310e4b95e0347e4e22f376151ab765e2dfd2c7",
18      "loop": "sema:Loop#mh:SHA-256:37
19        f953c0b21b9b0c72a639c4e1a54354faa71f07905531d271f6bbf40c341603",
20      "robustness": "sema:Robustness#mh:SHA-256:5304
21        c7824b76c6f308bb1a33ba6171d41fd061a5b104f21710bbb6c7a7c85b43"
22    }
23  },
24  "signature": [
25    "Check(Robustness)",
26    "Critique(Belief)"
27  ],
28  "mechanism": "Before finalizing a {{decision}} or output, the {{agent}} MUST
29    generate the strongest possible argument against its own conclusion. It
30    performs a {{check}} on the {{robustness}} of the claim and a {{critique}} of
31    the underlying {{belief}}. If the counter-argument exceeds a validity
32    threshold, the decision is discarded or revised. It prevents confirmation {{
33    cognitive_bias}}. Utilizes {{compatibility_check}}. For adversarial contexts,
34    see adversarial steelmanning.",
35  "gloss": "Post-decision adversarial check: revise if counter-argument exceeds
36    validity threshold",
37  "invariants": [
38    "Counter-argument scores at or above 'strength_threshold'.",
39    "The counter-argument identifies the specific claim it contradicts.",
40    "A passing critique is required before release."
41  ],
42  "preconditions": [
43    "A {{decision}} or output exists to challenge.",
44    "Claim is falsifiable"
45  ],
46  "postconditions": [
47    "{{critique}} generated and scored"
48  ],
49  "parameters": [
50    {
51      "name": "iteration_limit",
52      "type": "Integer",
53      "range": "[1, 5]",
54      "description": "Max strengthening attempts"
55    },
56    {
57      "name": "strength_threshold",
58      "type": "Float",
59      "range": "[0.7, 0.95]",
60      "description": "Min strength to pass as steelman"
61    }
62  ]
63 }

```



```

45     }
46 ],
47 "failure_modes": [
48     "Paralysis by analysis (stuck in {{critique}} {{loop}}).",
49     "Collusion: Proposer and Critic are same entity; susceptible to Strawman Waltz
      attack where agent generates weak counter-arguments to easily defeat them
      ."
50 ]
51 }

```

A.5 sema:OptimisticSolver#mh:SHA-256:f664c4e84cbf04e0c16b1fcac7c2662b4b35f4b474b76a2a47e129b36a3dc639

Note: This pattern was downgraded to **Tier 2** following adversarial analysis.

```

1  {
2    "dependencies": {
3      "composes_with": {
4        "atomic_bid": "sema:AtomicBid#mh:SHA-256:820
          c7c2788d8bfc50cb80d142201f63fc726823c60e34e1c8567e4f66c72c8c1",
5        "compensate": "sema:Compensate#mh:SHA-256:447
          fb91007e09d8674d66ab2698189026a2e9f70a2b66fbaaed406006fb8b8eb",
6        "compute_budget": "sema:ComputeBudget#mh:SHA-256:4091
          ac7c6677342920dc53808d02830e35c5849f2af1bb9ba7749c576d37478e",
7        "pathway_memory": "sema:PathwayMemory#mh:SHA-256:
          b15f1dd794db8b776de7646d2431466934bb901c3d443e0c92e85b58ad3c3ca5",
8        "reflexion": "sema:Reflexion#mh:SHA-256:
          b2b59e0b162b7c5f91b36d991fffb143331568be2d1dbc56218ccfccafd40dfc3"
9      },
10     "references": {
11       "parallel": "sema:Parallel#mh:SHA-256:
          d58d6696be11be09651e71307f0ea6f42bb45fac137ed985a61e475144f687a5",
12       "polymorphic_solver": "sema:PolymorphicSolver#mh:SHA-256:1362
          a5c7db1715b6683768b75720388810451c79127e566b8f1b4c27bb52391c",
13       "rigorous_solver": "sema:RigorousSolver#mh:SHA-256:1275832
          b5bb4e2ad2e682f32326f8a02f105df57d582be23afd1fb1a7cfae921"
14     },
15   },
16   "mechanism": "A high-velocity implementation of {{polymorphic_solver}} designed
      for efficient multi-agent coordination. Requires a {{parallel}} runtime (
      Actor Model with Mailboxes) to prevent serial deadlock. It explicitly couples
      the standard Solver lifecycle (Reason -> Solution) with the {{atomic_bid}}
      protocol. Unlike the base abstraction, this pattern MANDATES that the agent
      plan and execute in a single turn. It relies on {{reflexion}} and {{
      compensate}} for error correction rather than pre-action permission. Use {{
      compute_budget}} to bound resource consumption. Contrast with {{
      rigorous_solver}} which prioritizes safety over speed. A {{pathway_memory}}
      accumulates across runs so the optimistic route-selection converges on
      strategies that historically succeeded under similar conditions.",
17   "gloss": "High-velocity solver requiring parallel runtime",
18   "invariants": [
19     "Turn Atomicity: Must output [BID] and [TOOL] in the same response.",
20     "Non-Blocking: Cannot wait for Orchestrator approval on standard tasks."
21   ],
22   "preconditions": [
23     "Runtime supports Asynchronous/Parallel message delivery (Actor Model).",
24     "Message Bus is non-blocking (Mailbox pattern)."
25   ],
26   "failure_modes": [
27     "Over-Eager Execution: Solving the wrong problem efficiently because feedback
      was skipped.",
28     "Serial Deadlock: If deployed on a single-threaded (Talking Stick) engine,
      multiple atomic outputs will be dropped, halting the swarm."
29   ],

```

```

30 "extends": "sema:PolymorphicSolver#mh:SHA-256:1362
    a5c7db1715b6683768b75720388810451c79127e566b8f1b4c27bb52391c"
31 }

```

B Category Taxonomy

This appendix presents the complete taxonomic hierarchy of the Sema ontology.

The 456 patterns are organized into 12 category names across 13 layer-category paths and 4 fundamental layers:

Physics Layer (17 patterns) The immutable substrate.

- **Primitives** (16): Attractor, Causation, Conservation, Dampen, Decay, Distance, Entropy, Equilibrium, Gradient, Lock, Measurement, Mutex, MutualInformation, Noise, PhaseTransition, Reversibility.
- **Time** (1): CausalBarrier.

Infrastructure Layer (157 patterns) Operational constraints.

- **Data Structures** (96): AcceptSpec, Aesthetics, Anomaly, Artifact, Assessment, Assumption, Audit, Ballot, Belief, Boolean, Break, Cache, Card, Category, Chain, CognitiveBias, ConceptAnchor, Condition, Constraint, Context, Contract, Correlation, Criteria, Cyclic, DAG, Datum, Decision, Event, Exception, ExecutionManifest, FailureTrace, Forest, FrameSpec, Goal, Hierarchy, Hypothesis, Identity, Ledger, MECE, MechanisticDesignProposal, Message, Meta, Metric, MetricReading, Mode, Nature, Option, Outcome, Overlap, Parallel, PerformanceSignal, Permission, Plan, Probability, Problem, ProblemSpace, Prompt, Proposal, ProtoPack, Protocol, Prototype, Queue, Resource, Result, Risk, Role, RolloutManifest, RuleSet, Score, ScoringFunction, Sequence, Shard, Signal, Skeleton, Snapshot, Solution, SolverManifest, Spec, State, Status, Step, Stream, StyleSpec, Subject, Summary, System, Task, Tension, Topology, Trait, Transition, Tree, Value, Variable, Vector, Work.
- **Primitives** (52): Act, Actor, Aggregate, Backoff, Branch, Budget, Care, Check, CircuitBreaker, Combine, Compare, Compensate, Compress, Cooldown, EntropyPump, Equivalence, ExponentialBackoff, FailClosed, Feedback, FeedbackSignal, Gate, Greet, Heartbeat, Hysteresis, IdempotentWrite, Incongruity, Judge, Loop, Monitor, MonitorReport, NegativeProof, Observe, ParetoFront, Probe, Quorum, Rank, ReAttempt, Route, Sandbox, Search, Select, Sign, StateAudit, StateSnapshot, StateTransition, TaskLifecycle, Throttle, TimeWarpLog, ToolInvoke, Trace, TriGate, Warmup.
- **Verification** (9): AuditTrail, CompatibilityCheck, ExplainBeacon, HumanApprove, InputGuard, OathBind, OutputGuard, SpotAudit, Validate.

Mind Layer (179 patterns) Hybrid cognition—always decomposable and delegatable.

- **Inference** (22): BaseRateInclude, BayesUpdate, BreadthGovernor, ConfidenceCalibrate, ConfirmationBlock, ContextFirst, EpistemicCalibrate, HackDetect, HindsightBlock, LayeredCheck, NormCheck, NormativeJudge, OntologyAdapt, ProphetFanOut, RegimeSense, ScopeFreeze, SemanticTabu, SourceEvaluate, SurprisalUpdate, SurvivorCorrect, TemporalEnsembleForecasting, TruthseekingProtocol.
- **Memory** (15): BeliefTracking, ChunkMerge, ContextCompress, CurriculumReplay, ExperienceSharding, HolographicShard, LatentAttachment, LocalizedLearning, PathwayMemory, Proprioception, RetrievalAugment, Scratchpad, SelfReminder, SimulationTrace, TraceBelief.
- **Reasoning** (61): Abduction, BackwardChain, Bisect, ChainOfThought, CiteBack, CognitiveEcho, CollaborativeWritingProtocol, ConceptualDecomposition, ConstructOntology, Critique, Decompose, DecompositionGate, Deduction, DeepResearch, Dialectic, Eliminate, Estimate, EthicalReasoningProtocol, Expansive, ExtendedThinking, Fermi, FirstPrinciples, FrameError, Generalize, GraphOfThought, HeuristicSnap, HumanEmulatorProtocol, Induction, Interpret, Invert, LeastToMost, LivedProof, MetaPrompt, Parsimony, PatternDiscovery, PatternEmergence, ReAct, Realizable, Reason, RecursionDive, RecursiveRootCause, Refine, Reflexion, Reframe, RequestFraming, SelfConsistency, SkeletonOfThought, SocraticLoop, Specialize, SteelmanCheck, StepBack, StrategicReading, Summarize, Synthesis, Think, Translate, TreeOfThoughts, Uncertain, Understand, Verification, WhyClimb.

- **Strategy** (81): AdversarialSteel, Agent, AnalogyBridge, AntifragileInversion, BeamSearch, Bubble, Build, CapacityPressure, CommitmentDevice, Compose, ComputeBudget, ConceptBlend, ConstraintFirst, ContingencyPlan, Creative, CreativeBlend, Crystallize, Deep, Defer, DepthGovernor, DesignArchitect, DiscoveryProtocol, DogfoodFirst, EmpathySim, EmpiricalTest, EpistemicROI, EventReact, Experiment, ExploreExploit, Falsification, FractalIntelligence, HypothesisEngine, HypothesisLadder, Jester, Kairos, LatentWander, LateralOptimization, ManifestPlanning, MarginalValueRule, MentalSim, MetaCheck, MetaProtocols, MintWhenFriction, NoiseInjection, Novelty, OODA, OpportunityCost, OptimalStop, Optimize, PURE, PUREBrainstorming, PURECheck, PUREOptimization, Parallelize, PerspectiveEnsemble, PolymorphicSolver, PreMortem, Prioritize, ProblemFramer, RedTeam, Reflex, RegretMinimization, RepresentationSwap, Retry, RigorousSolver, Roadmap, RootSolver, SacrificialProbe, Satisfice, Silence, Simulation, Solver, SteelmanFirst, Strategy, SunkCostIgnore, TensionHold, ThinSlice, TimeboxThink, TradeOff, UncertaintyMap, WorldReversible.

Society Layer (103 patterns) Multi-agent coordination.

- **Coordination** (12): Compromise, Consensus, ConsensusFinder, Delegate, Disband, Elect, IdentityHandshake, LazyConsensus, OntologyHandshake, Rally, Resonate, Vote.
- **Economics** (9): AtomicBid, AttentionMarkets, Award, Bid, ContinuousResourceAuction, ExchangeRate, Gardener, ValuePeg, Yield.
- **Governance** (7): AnchorDrop, Constitution, DocumentedOverride, Responsibility, SolverTree, UniversalSolverTree, WorldTransparent.
- **Protocols** (75): AdversarialProof, AgentDiscover, AgentProtocol, AgentSandbox, AmbiguityResolution, Axiom, BearerToken, BoundedTask, Canary, ConfusedDeputy, ContextSwitch, CounterfactualAnchor, DataMinimization, DeliberativeAlign, Deploy, Discover, DissentSeek, DriftWatch, EbbFlowSync, EjectionSeat, EvaluatorOptimizer, ExpiringToken, FabricSharding, FeatureFlag, GenealogicalTrace, GlacialVault, Global, GracefulDegradation, Handoff, HeldRelease, IntentGap, InternalConsistency, InvariantFilter, LatticeCommit, MemeticSeed, ModestClaim, MonotonicCounter, Nucleate, OptimisticSolver, Oracle, OrchestrationLoop, OsmoticFilter, PatternSketch, PermissionEscalate, PhasedRefinement, PromiseGraph, PromptChain, PropheticQuorum, QuorumPulse, RealizationProtocol, ReceptivityGate, ReversibilityCheck, Robustness, Rollout, RolloutWatch, RootHashGossip, ShoutWhisper, SignalReflection, SolverNode, SomaticMarker, SpectralTune, StateLock, Stigmergy, StructuralCoaching, SynergisticMode, Taper, ThreeLevelCollision, TieredAccess, ToolDiscovery, TranslationProxy, UniqueHandle, UptakeAsGround, UptakeOverTimestamp, WorkerMode, Workflow.